Machine-Checked Correctness for Djex and Leant

A Lean 4 formalization feasibility study, proof decomposition, and certification architecture

This report examines the current Djex, Leant, and Lean 4 implementations; identifies correctness claims that can be stated precisely; separates easy positive guarantees from substantially harder negative guarantees; and proposes a staged architecture in which the existing Haskell programs remain fast, untrusted certificate producers while small Lean components check all externally visible claims.

Prepared for Vladimir Reshetnikov

10 August 2026

Repository revisions are pinned in Chapter 2.

Status, 14 August 2026: since this snapshot, Djex has grown the untrusted-producer half of the staged architecture proposed here: a bounded source-certificate table with atomic term-graph association and carrier-aware canonical fingerprints. No Lean-side checker exists yet; every certification claim in this report remains future work. A companion study, *Rewriting Leant and Djex in Lean 4* (`docs/Leant_Djex_Lean4_Rewrite_Analysis/`, 15 August 2026), asks the complementary question — not what Lean could certify about the existing Haskell implementation, but whether the Lean-facing parts should be reimplemented in Lean itself.

Abstract

The most valuable correctness properties of Djex and Leant are formalizable in Lean 4, but they do not form one monolithic theorem. They decompose into at least five layers: well-typedness of generated core terms; soundness and completeness of the Djinn/LJT decision procedure; honest accounting for bounded Exference search; adequacy of the Lean-to-Djex abstraction; and acceptance of reconstructed terms by a particular Lean kernel and environment.

The positive direction is unusually favorable. A generated candidate need not come with a proof that the synthesizer is correct: it can be treated as hostile input and checked independently. Djinn already has a structurally separate proof checker, Exference already reconstructs and checks candidate types, and Leant already re-elaborates each rendered candidate against the exact target before displaying it. A Lean formalization of the small Djinn proof checker, followed by proof-carrying candidate export and module replay, can therefore produce a strong result without verifying the Haskell search implementation.

The negative direction is fundamentally different. Search exhaustion alone is not a proof that a Lean type has no inhabitant. A sound negative statement requires (i) a formally defined term language or calculus, (ii) a checked certificate that the corresponding sequent is underivable, and (iii) a one-way adequacy theorem saying that every source inhabitant permitted by the advertised scope would induce a derivation in the abstract calculus. Exact equivalence is unnecessary: an over-approximation is sufficient. This observation makes useful partial certification attainable, but it also exposes an important scope boundary. A provider-free Djinn refutation establishes non-inhabitance in the structural calculus; it does not by itself exclude an arbitrary constant already present in the ambient Lean environment, nor does it exclude Haskell bottom.

The recommended near-term architecture is therefore certificate-based. Positive results carry a checked proof term. Negative propositional results carry either a complete LJT refutation DAG or, preferably, a finite Kripke countermodel checked by Lean. Leant's projection emits proof-carrying abstraction evidence rather than boolean completeness flags. The first fully certified negative fragment should be the pure, closed structural grammar of arrows, products, sums, top, bottom, and opaque parameters, with a precise environment policy. Rank-N instantiation, arbitrary inductive families, type classes, recursive providers, and full ambient-environment non-inhabitance should be added only when their one-way adequacy theorems exist.

Contents

Chapter 1

Executive assessment

1.1 Bottom line

A substantial and useful formalization is feasible. The right first objective is not "verify all of Djex's Haskell code," but "make every externally visible claim checkable by a small Lean certificate checker."

The existing code is already close to the right architectural shape. Djinn separates proof search from proof checking. Exference separates candidate generation from contextual type reconstruction. Leant treats synthesized text as untrusted and asks Lean to elaborate it against the exact goal. These boundaries can be strengthened incrementally rather than replaced.

The work divides sharply by claim strength:

- **Positive candidate correctness** is the easiest and highest-value theorem. It is possible to certify that every displayed term was accepted at the requested type by a kernel-enabled Lean process, and to attach a replayable declaration artifact.
- **Core-calculus correctness** is also tractable. The current Djinn formula and term languages are small enough for a direct deep embedding, executable proof checker, and machine-checked soundness theorem.
- **Propositional negative certificates** are tractable without verifying Haskell search. A finite Kripke countermodel or a complete refutation tree can be checked in Lean and proves meta-level non-derivability.
- **The exact Djinn decision procedure** is formalizable but more involved. The proof must cover the implementation's indexing, pruning, proof-term construction, fairness policy, and explicit budgets, not merely quote the paper theorem for G4ip/LJT.
- **Full Leant negative correctness for arbitrary Lean targets and environments** is a research-scale goal. It requires a formal abstraction theorem for Lean terms, a precise account of permitted global constants, and eventually normalization or parametricity results for richer fragments.
- **Bit-for-bit refinement of the current GHC-compiled Haskell executables** is possible in principle but is a poor first target. Laziness, exceptions, partial functions, finite machine integers, library behavior, and the gap between GHC semantics and a Lean model would dominate the project while adding little immediate assurance beyond certificate checking.

Candidate claim	Precise formal target	Feasibility	Recommended treatment
Displayed LEANT term has the requested type	A fixed-option Lean kernel accepts a declaration whose type is the exact elaborated goal and whose value is the displayed term	**strong**	Harden the existing gate; emit and replay a named declaration; record dependencies and toolchain hash
DJINN proof checker is sound	Successful executable checking implies a declarative typing derivation in the core calculus	**strong**	First Lean package and first theorem
DJINN search returns only proofs	Every emitted search term passes the formal checker	**strong**	Obtain immediately by certificate checking; separately prove generator refinement later
DJINN decides propositional inhabitation	Termination plus soundness and completeness for the exact implemented formula fragment	**medium**	Formalize after the checker, using corrected G4ip termination arguments and implementation-specific lemmas
Negative IPC verdict is sound	A checked refutation certificate implies no derivation in the formal IPC calculus	**strong**	Prefer a verified finite Kripke countermodel; retain refutation DAG as an alternative
EXFERENCE candidates are well typed	Its contextual checker accepts only terms with the claimed type and solved constraints	**medium**	Formalize checker, unifier, and constraint-solver soundness; do not claim complete search
EXFERENCE completion status is honest	Each status corresponds to a proved operational condition: exhaustion, bound, pruning, or identifier-space exhaustion	**strong**	Model finite scheduler and counters; property tests become refinement tests
Leant projection is negative-sound	Every permitted Lean inhabitant yields a derivation of the projected sequent	**medium–hard**	Start with a small pure structural fragment and proof-carrying projection evidence
"No Lean term exists" in the full live environment	No kernel term using any available declaration has the target type	**weak**	Do not use as the first theorem. State a restricted language/environment claim unless a closed environment model is certified
Exact GHC executable refines Lean specification	All observable results of compiled Haskell agree with the formal model	**weak**	Optional final layer; certificate checking gives much better assurance per unit of formalization

Table 1.1: The proposed correctness claims, ordered by practical attainability rather than by logical strength.

1.2 Feasibility matrix

1.3 The recommended first end-to-end result

A small but genuinely strong first milestone is:

1. Define the Djinn core formula and proof-term syntax in Lean using de Bruijn indices.
2. Define a declarative typing relation.
3. Port the independent proof checker, including unification and constructor metadata checks.
4. Prove that successful checking yields a typing derivation.
5. Add a Djex mode that serializes each generated proof term and formula as a certificate.
6. In CI and optionally at runtime, ask the Lean checker to validate the certificate before rendering the Haskell or Lean term.

This proves a meaningful theorem about the actual producer/checker boundary while leaving search order, heuristics, and Haskell execution outside the trusted computing base. It also creates the infrastructure needed for negative certificates and Leant projection evidence.

1.4 Two findings that should change the specification

Finding. For a sound negative result, Leant's abstraction need not be an isomorphism. It only needs to be an over-approximation: every source inhabitant in the advertised scope must map to an abstract derivation. This one-way theorem is sufficient by contraposition and is materially easier than full semantic equivalence.

Scope warning. A provider-free refutation does not exclude arbitrary constants in the ambient Lean environment. The current user-facing phrase "no closed term of this polymorphic type exists" should be read as a structural-calculus claim unless provider/environment closure is separately certified. The report proposes exact replacement labels in Chapter 17.

A third implementation hardening item is immediate: the verification command must force `debug.skipKernelTC = false`. Lean's checked declaration path explicitly bypasses kernel type checking when this option is enabled, while Leant exposes persistent option setting. A dedicated verifier process with fixed options is preferable to relying on mutable REPL state.

Chapter 2

Scope, revisions, and method

2.1 Pinned source revisions

This report inspected the following repository states on 10 August 2026:

Repository	Revision	Role in this report
Djex	`f50b5eac86726c7530d95e48c27101b7c0fa25c9`	Shared synthesis types, Djinn/LJT, independent proof checking, Exference checking and bounded search
Leant	`0160237e4bdf6e342a55f5a86ce5d5292728024f`	Lean serializer/projection, provider discovery, engine orchestration, rendering, candidate verification, and reporting
Lean 4	`1d2dde6d64cb3da989bab118f4d386eb55237fe5`	Kernel declaration checking, option boundary, module replay via `leanchecker`, and the target implementation for generated artifacts
`lean4lean`	`1a16b72d2e35932a82aa501beb29ef2c3d072580`	Optional independent checker and reusable formal infrastructure for Lean expressions, environments, and type checking

Table 2.1: Pinned revisions. Leant's `lib/Djex` submodule points exactly to the Djex revision shown above.

The repositories are under active development. All implementation observations in this report refer to these hashes, not to an unspecified future `main` or `master` branch [1, 2, 3].

2.2 Files and boundaries examined

The review concentrated on the following modules:

- Djex: `synthesis/.../Type.hs`; `Djinn/Internal/LJTFormula.hs`; `Djinn/Internal/LJT.hs`; `Djinn/Internal/ProofCheck.hs`; `Djinn/Core.hs`; `Exference/Core/ExpressionCheck.hs`; `Exference/Core/Internal/SearchControl.hs`; and `Exference/Core/Internal/Exference.hs`.

- LEANT: `Leant/Synth/Fragment.hs`; `Leant/Synth/Engine.hs`; `Leant/Synth/Render.hs`; and the synthesis orchestration and verification code in `src/Main.hs`.
- LEAN 4: `src/Lean/AddDecl.lean` and `src/LeanChecker.lean`, plus the repository's current kernel-hardening issue and fix discussed in Section 11.5.

The review was architectural and proof-oriented rather than a line-by-line audit of every helper. It sought theorem boundaries, trusted inputs, explicit approximation flags, and places where a runtime check can be replaced by a proof-producing or proof-checking interface.

2.3 Methodological principle

The analysis uses a deliberately asymmetric trust model:

> **Generators may be arbitrarily complicated and untrusted. Checkers should be small, total, deterministic, and proved sound.**

This is the same broad architecture used by verified SAT tooling: a high-performance external solver produces a certificate, and a small checker with a soundness theorem imports the result. LeanSAT's verified LRAT checker is a direct precedent inside the Lean ecosystem; it has since moved into Lean core as part of `Std.Tactic.BVDecide` [7]. The proposed DJEX/LEANT architecture applies the same separation to proof synthesis and negative inhabitation evidence.

Chapter 3

What exactly should be proved?

3.1 Correctness is not one predicate

A statement such as "Djex is correct" can refer to several non-equivalent properties:

1. **Core typing soundness:** every generated proof term is well typed in a formal core calculus.
2. **Search soundness:** every successful branch corresponds to a valid derivation.
3. **Search completeness:** if a derivation exists in the target calculus, the unbounded decision procedure finds one.
4. **Termination:** the unbounded decision procedure returns on every well-formed input in the advertised fragment.
5. **Bound honesty:** a bounded search never reports logical impossibility merely because fuel, queue space, wall-clock time, or an identifier budget ran out.
6. **Translation soundness for positive results:** a core proof can be reconstructed as a source-language term of the original type.
7. **Abstraction adequacy for negative results:** every permitted source-language inhabitant would induce a core derivation.
8. **Kernel acceptance:** a particular Lean kernel, in a particular environment and with kernel checking enabled, accepts the generated declaration.
9. **Axiom policy:** the accepted term depends only on an allowed set of axioms and declarations.
10. **Implementation refinement:** the compiled Haskell executable implements the formal specification for all relevant observations.

These properties should be named separately in APIs, certificate formats, documentation, and user-facing output. A single boolean named `sound` cannot carry enough information.

3.2 Positive evidence and negative evidence are asymmetric

For a positive answer, an untrusted engine may output any string. If Lean accepts that string at the exact target type, engine correctness is no longer required for the displayed theorem. The engine affects completeness, ranking, and performance, but not logical soundness.

For a negative answer, there is no analogous one-line kernel check. The absence of a generated term is not evidence unless the search space and the source-to-search abstraction are both formally delimited. This asymmetry should drive the whole design.

3.3 Meta-level non-derivability is not object-level negation

Let $\Gamma \vdash A$ mean that the formal calculus has a derivation of A from Γ. A decision procedure may return a Lean proof of

$$\neg\, \mathsf{Derives}(\Gamma, A).$$

This is a proposition about syntax and derivations inside the metatheory. It is not generally a term of the object formula $A \to \bot$.

Intuitionistic logic makes the distinction unavoidable. There are formulas for which neither A nor $\neg A$ is derivable. Excluded middle with an opaque atom is the canonical example. Therefore a successful refutation certificate should be described as *non-derivability in a named calculus*, not as a proof of the target's negation.

For some closed polymorphic types, an object-level refutation is available. For example, a hypothetical inhabitant of $\forall a\, b : \mathsf{Type}, a \to b$ can be instantiated at $a = \mathsf{Unit}$ and $b = \mathsf{Empty}$. Such special cases are useful, but they do not replace the general meta-level statement.

3.4 Haskell bottom changes the meaning of "uninhabited"

In ordinary non-strict Haskell semantics, every lifted type admits nontermination or `undefined`. Consequently, Djinn's negative result cannot literally mean that no Haskell expression inhabits the type. The meaningful claim is instead one of the following:

- no total closed normal form exists in the pure generated lambda calculus;
- no term exists in the explicitly defined Djinn core syntax;
- no parametric total term exists in a System-F-like model of the supported Haskell fragment.

The Lean formalization should choose one of these statements explicitly. The first implementation should use the syntactic core statement because it is exact, decidable, and independent of disputed operational notions of totality.

3.5 The minimum theorem for a sound negative projection

Suppose E is a source environment, T a source target, and (Γ, A) the abstract sequent produced by the translator. Define $\mathsf{SourceTerm}_L(E, T)$ to mean "a source term of type T built from the permitted language and constant policy L." The essential theorem is one-way:

$$\mathsf{Abstracts}(E, T, L, \Gamma, A) \quad \Longrightarrow \quad (\mathsf{SourceTerm}_L(E, T) \to \mathsf{Derives}(\Gamma, A)).$$

Then

$$\neg\mathsf{Derives}(\Gamma, A) \quad \Longrightarrow \quad \neg\mathsf{SourceTerm}_L(E, T).$$

The abstract calculus may have *more* derivations than the source language. This is safe: failure in an over-approximation implies failure in the source. What is unsafe is an under-approximation that discards a source construction and thereby makes the abstract formula artificially difficult to inhabit.

This observation gives a precise interpretation to many current approximation flags. A translation plan is negative-safe exactly when it carries a proof of the implication above, not merely when a boolean heuristic says that no depth marker or unsafe atom was encountered.

Chapter 4

Current architecture and existing safeguards

4.1 Djex: two search engines behind checked boundaries

Djex is not one algorithm. It contains two engines with intentionally different contracts:

- Djinn is a decision-procedure-oriented synthesizer for an intuitionistic structural calculus. Its formula language contains products, constructor-tagged sums, nominal empty types, implication, and atomic propositions. Its term language is a typed proof-term representation with variables, lambdas, application, tuples, splitting, injections, case analysis, and projections.
- Exference is a heuristic ranked synthesizer over a richer polymorphic and constrained type language. It has explicit step, queue, depth, and identifier-space bounds. Its useful correctness target is soundness of emitted candidates and honesty of completion status, not global completeness.

The shared `haskell-synthesis` layer supplies names, source types, constraints, capture-avoiding substitution, and validation. This is a natural future location for a versioned certificate schema, but the two engines should retain distinct search specifications.

4.1.1 Djinn's independent proof checker

A particularly valuable boundary is `Djinn/Internal/ProofCheck.hs`. Search produces a term; the checker reconstructs a proof type, generates unification constraints, solves them with an occurs check, and compares the result with the expected formula. The public orchestration rejects a candidate that does not pass this checker before converting it into source syntax.

This means the first formal proof need not establish that every branch in `LJT.hs` is implemented correctly. It can establish that the independent checker accepts only valid terms. The Haskell checker then becomes a reference implementation and test oracle; a Lean checker becomes the trusted one.

4.1.2 Djinn's current honesty lattice

The current Djinn API already distinguishes several reasons why absence of a candidate is not a refutation. A search outcome records generated proofs, whether search was exhausted, and remaining budget. Translation and instantiation plans also carry whether negative evidence remains sound. The orchestration downgrades to an undecided result when, among other conditions, a budget stopped the search or an approximation invalidated negative evidence.

This is conceptually right. The formal replacement is to make the evidence type indexed by the claim:

Listing 4.1: Evidence should be typed by what it proves, not represented by a boolean.

```
inductive SearchEvidence (s : Sequent) : Type
  | witness    (d : Derives s)
  | refutation (h : Not (Derives s))
  | bounded    (r : BoundReport)
  | approximate (r : ApproximationReport)
```

Only the `refutation` constructor may support an uninhabited verdict. A producer cannot manufacture it directly; it must provide a certificate accepted by the Lean checker.

4.2 Exference: bounded generation plus contextual checking

Exference's current implementation has a similarly useful separation. Candidate generation is heuristic and resource-bounded. A distinct expression checker reconstructs types under local binders, global function bindings, deconstructors, rigid skolems, class constraints, and substitutions. Only privately wrapped validated candidates cross the core API boundary. Simplified and unsimplified forms are checked rather than trusting transformation passes.

Its completion status is already richer than a single exhausted bit. The implementation distinguishes ordinary exhaustion, a step limit, pruning, and identifier-space exhaustion. These distinctions should survive formalization. In particular:

- `Exhausted` may mean the finite configured search graph was fully consumed.
- `StepLimitReached` is operational only and must never imply non-inhabitance.
- `Pruned` records deliberate incompleteness.
- `IdentifierSpaceExhausted` records an implementation resource failure rather than a logical result.

A formal model can prove that each constructor is returned only under its stated condition. It should not attempt to turn the heuristic engine into a complete decision procedure.

4.3 Leant's current synthesis pipeline

At the inspected revision, the relevant pipeline is approximately:

1. Lean elaborates the user's goal and a generated Lean metaprogram serializes a structural fragment as an S-expression.
2. Haskell parses that fragment, records refusal and approximation reasons, and optionally discovers a bounded provider inventory from the live environment.
3. Djinn or Exference searches one or more structural/provider lanes.
4. Djex returns candidate proof terms or a structured negative/bounded outcome.
5. Leant's renderer produces one or more Lean textual variants for each engine candidate.
6. Each variant is submitted as an `example : (goal) := (term)` declaration. A candidate is kept only when the backend reports no errors, no fatal result, and no newly introduced sorry.
7. Survivors are displayed and optionally bound into the session.

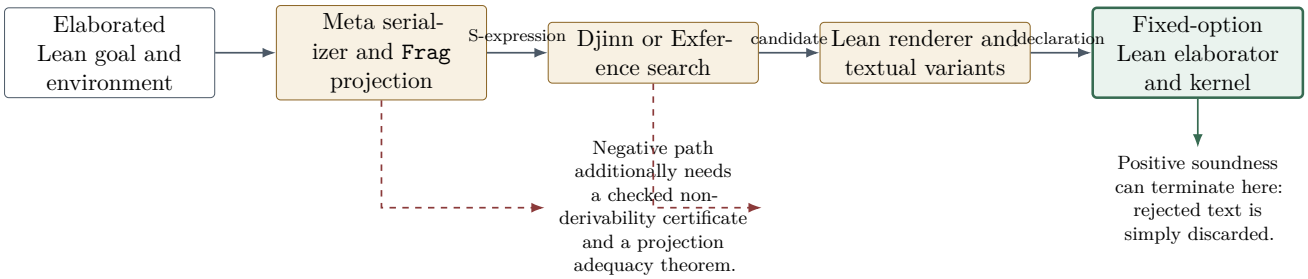

Figure 4.1: Current and proposed trust boundaries. Search and rendering may remain untrusted on the positive path. The negative path requires additional formal evidence.

4.3.1 Positive candidates are already treated defensively

The implementation's strongest existing design rule is that only backend-verified candidates are shown. Textual ambiguity in universes, implicit binders, constructor syntax, and provider application is handled by generating variants and keeping the first one Lean accepts. This makes renderer bugs incompleteness bugs rather than soundness bugs.

The guarantee is nevertheless conditional on the verifier configuration and environment. The verification process must ensure:

- kernel type checking is not disabled;
- no synthetic or explicit sorry entered the generated declaration;
- the checked target is byte-for-byte or expression-for-expression the intended elaborated target;
- the candidate is checked in the intended environment, not a stale or partially replayed one;
- imported axioms and dependencies satisfy the advertised policy.

4.3.2 Negative reporting is careful, but the scope is still too broad

The current code distinguishes a sound `SynthRefuted True` result from opaque/unsafe cases and bounded misses. It also delays a provider-free refutation while trying provider lanes, and it describes timeouts as "no answer, not a verdict." These are good safeguards.

However, after provider lanes fail, the code restores the provider-free structural refutation and reports that no closed term of the polymorphic type exists. The provider inventory is an optimization and is explicitly bounded and filtered. Therefore this message cannot, without an additional closure theorem, quantify over every constant in the live Lean environment. A user may have declared an axiom or definition of exactly the target type outside the selected inventory.

The formal result should instead expose the provider set or term-language policy in the proposition:

$$\neg \mathsf{TermUsing}(E, P, T),$$

where P is the certified set of allowed providers. A full-environment claim requires proof that every usable global constant is represented in P, or a syntactic rule that forbids all global constants except those explicitly listed.

4.4 Lean's declaration-checking boundary

At the inspected Lean revision, `Lean/AddDecl.lean` routes a declaration through kernel checking unless `debug.skipKernelTC` is enabled [4]. The same file separately warns about declarations containing sorry.

These are distinct mechanisms: a warning policy is not a substitute for kernel checking, and kernel checking does not by itself enforce an axiom policy.

Leant exposes persistent option setting. The candidate verification program should therefore override mutable state locally:

Listing 4.2: Recommended fixed verifier wrapper.

```
set_option debug.skipKernelTC false in
set_option warn.sorry true in
set_option autoImplicit true in
noncomputable example : (TARGET) := (CANDIDATE)
```

A stronger design runs this declaration in a separate verifier backend whose options cannot be modified by the interactive session. The main REPL may remain flexible; the verifier should be deliberately boring.

4.5 Replay and independent checking

Lean's `leanchecker` replays declarations from compiled modules through the kernel [5]. Its own source accurately describes it as a detector for environment hacking rather than an external verifier: it shares the implementation and kernel. It is still valuable as a second execution path and as a CI artifact checker.

The `lean4lean` project supplies a mostly pure Lean reimplementation of Lean's environment and type-checking machinery, with a CLI intended to mirror `leanchecker` [6]. It is a useful optional second checker and a source of reusable definitions. It should not be treated as magically independent: code derived from the same implementation can reproduce the same conceptual bug. The best assurance comes from diversity of checker structure and from small domain-specific certificates whose checker is far simpler than the full Lean kernel.

Chapter 5

Proposed theorem decomposition

5.1 The dependency graph

The desired claims form a directed graph rather than a single proof. The central dependencies are shown in Figure 5.1.

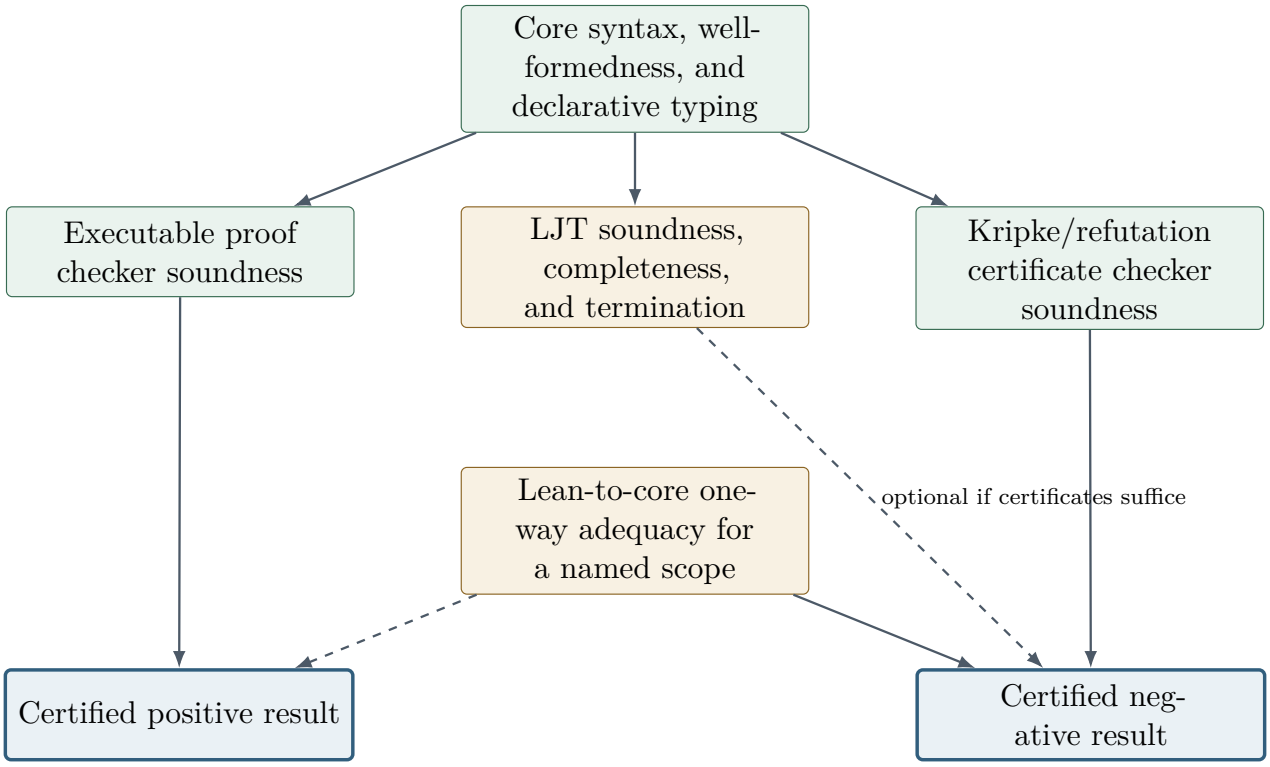

Figure 5.1: Theorem dependency graph. Positive certification can stop at independent checking. Negative certification requires both non-derivability evidence and a one-way source abstraction theorem.

5.2 Layer A: core syntax and declarative typing

Define a small, implementation-independent calculus. The first version should use de Bruijn indices and a normalized binary grammar:

$$A, B ::= p \mid \mathbf{0} \mid \mathbf{1} \mid A \times B \mid A + B \mid A \to B.$$

The current n-ary conjunction and constructor-tagged sum can be represented directly or normalized to binary syntax with a proved equivalence. Keeping constructor metadata in certificates is useful for reconstructing source terms, but logical non-derivability can ignore cosmetic constructor names after validating arities and distinctness.

The declarative relation

$$\Gamma \vdash t : A$$

should be an inductive family whose constructors correspond to variable, lambda, application, product introduction/elimination, sum introduction/elimination, top introduction, and bottom elimination. This relation is the specification against which both Djinn and Exference checkers are proved.

5.3 Layer B: executable checker soundness

The main theorem has the shape:

Listing 5.1: First central theorem.

```
theorem check_sound
    (h : checkTerm env expected term = Except.ok unit) :
    HasType env term expected
```

The checker may infer a type with metavariables and then unify it with the expected type. The proof therefore decomposes into:

1. well-formed substitutions preserve types;
2. the occurs check prevents cyclic substitutions;
3. unification success implies equality after applying the produced substitution;
4. inferred constraints are sound with respect to declarative typing;
5. solving all constraints and matching the expected formula yields the final derivation.

Checker completeness is useful for regression testing and for avoiding rejected valid certificates, but it is not required for logical soundness. It can be postponed.

5.4 Layer C: positive reconstruction

There are two possible positive theorems:

Core positive theorem
A checked certificate denotes a well-typed term in the formal core calculus.

Source positive theorem
The rendered source term elaborates to the exact requested type in the target environment.

The second is already established dynamically by Leant's final gate, provided the verifier is hardened. A formal proof that rendering preserves typing is optional; it would improve completeness and diagnostics but is not needed to protect soundness.

For durable evidence, Leant should emit a named declaration module and a manifest containing the goal expression hash, candidate source, elaborated declaration name, imported modules, Lean revision, options, and axiom dependency set. The module can be replayed by the official kernel and optionally by an independent checker.

5.5 Layer D: negative certificate soundness

A negative certificate checker proves one of:

$$\mathsf{verifyRefutation}(s, c) = \mathsf{true} \Longrightarrow \neg\mathsf{Derives}(s),$$

or

$$\mathsf{verifyCountermodel}(A, m) = \mathsf{true} \Longrightarrow \neg\mathsf{Derives}([], A).$$

The producer may obtain the certificate by any method: the existing LJT search, a separate model finder, SAT/SMT search, exhaustive enumeration, or a future learned heuristic. Only the checker's theorem enters the trusted story.

5.6 Layer E: source abstraction adequacy

The projection theorem should be indexed by an explicit scope:

Listing 5.2: One-way adequacy, sufficient for negative results.

```
structure Scope where
  allowedConstants : Name -> Prop
  allowRecursion    : Bool
  allowClassical    : Bool
  allowUnsafe       : Bool

structure ExactAbstraction
    (env : SourceEnv) (target : SourceType)
    (scope : Scope) (seq : Sequent) : Prop where
  abstractTerm :
    SourceHasType env scope term target -> Derives seq
```

The name `ExactAbstraction` may be replaced by `NegativeSoundAbstraction`; exactness is stronger than necessary. The important point is that the evidence is a proposition with proof fields, not a boolean computed by Haskell.

5.7 Layer F: full implementation refinement

Only after the certificate architecture is in place is it useful to prove that the Haskell producer always emits a valid certificate when it claims success. Such a theorem improves robustness and can eliminate runtime checker failures, but it is not required for external trust. A compiler-level theorem for the exact GHC executable is a separate project and should not block the earlier layers.

Chapter 6

Certifying positive results

6.1 A positive result can bypass search correctness

Suppose the engine proposes a candidate e for target T. The strongest practical positive contract is:

$$\mathsf{AcceptedByKernel}(E, e, T).$$

No theorem about the producer is needed. A malformed internal proof, a renderer capture bug, a wrong universe guess, or an invalid provider application merely causes rejection.

This principle should be reflected in the public result type:

Listing 6.1: A proof-carrying positive result.

```
structure VerifiedCandidate where
  source        : String
  declaration   : Name
  targetHash    : UInt64
  envFingerprint : ByteArray
  moduleArtifact : FilePath
  dependencies  : Array Name
```

The Haskell side should not be able to construct a `VerifiedCandidate` directly. It should receive one only from a verifier process that has successfully elaborated, kernel-checked, and persisted the declaration.

6.2 Hardening Leant's existing verifier

The inspected implementation already rejects errors, fatal backend results, and newly introduced sorries. The following changes would turn that good runtime check into a sharply specified boundary.

6.2.1 Use a dedicated verifier backend

A separate backend process should load the same module/environment snapshot but accept only a tiny verification protocol. Its options are fixed and not inherited from interactive `:set` commands. In particular it should force:

- `debug.skipKernelTC = false`;
- `warn.sorry = true`;
- a fixed trust level and heartbeat policy;

- no unsafe environment mutation APIs from the submitted snippet;
- a fresh collision-resistant declaration name.

The REPL backend remains responsible for exploration; the verifier backend is responsible for evidence.

6.2.2 Check an expression, not only a string convention

The verification request should carry the already elaborated target expression or a stable serialized target identity, not only a reprinted goal string. The verifier can elaborate the declaration and compare its type with the original target by definitional equality in the intended environment. This prevents accidental drift caused by pretty-printing, auto-implicit rebinding, namespace changes, or parser ambiguities.

A robust protocol is:

1. Leant asks Lean to assign a stable target identifier and environment fingerprint.
2. Candidate source is elaborated under that target.
3. The resulting declaration is added with kernel checking enabled.
4. The verifier reports the declaration name, type expression digest, value digest, and collected axioms.
5. The caller verifies that the target/environment fingerprints match the request.

6.2.3 Turn axiom use into an explicit policy

Kernel acceptance means type correctness relative to the current environment. It does not mean axiom freedom. A candidate may depend on imported axioms, quotient soundness assumptions, classical choice, user axioms, or previous declarations containing sorry.

Leant should report at least one of:

- **kernel-checked, unrestricted dependencies**;
- **kernel-checked, dependencies listed**;
- **kernel-checked, dependencies within allow-list**;
- **kernel-checked and axiom-free relative to imported theorem declarations**, when that claim is actually verified.

Lean's existing axiom-collection utilities can provide the dependency set. The manifest should distinguish primitive trusted axioms from user declarations and from sorry-derived declarations.

6.2.4 Persist a replayable artifact

An ephemeral `example` proves acceptance only during that backend interaction. A stronger artifact mode emits:

- a small `.lean` source module containing the candidate theorem or definition;
- the corresponding `.olean`;
- a JSON or S-expression manifest with source hashes and options;
- the certificate used by the core checker, if available.

CI can run the ordinary compiler, `leanchecker -fresh` where applicable, and `lean4lean`. The source module remains the durable human-readable proof object.

6.3 Formalizing Djinn's proof checker

6.3.1 Why this is the best first formal target

The checker is small enough to understand globally, independent of proof search, and already used as a defensive boundary. A Lean port therefore gives immediate value and a clean refinement target for the existing Haskell implementation.

The formal checker should initially normalize away implementation conveniences:

- use de Bruijn indices instead of globally fresh symbols;
- represent binary products/sums first, then prove normalization of n-ary forms;
- separate well-formed constructor metadata from typing;
- use an explicit finite map for metavariable substitutions;
- return structured errors only after the soundness theorem is stable.

6.3.2 Declarative typing rules

For a context Γ, representative rules are:

$$\frac{i : A \in \Gamma}{\Gamma \vdash \mathsf{var}\ i : A} \qquad \frac{A :: \Gamma \vdash t : B}{\Gamma \vdash \lambda t : A \to B} \qquad \frac{\Gamma \vdash f : A \to B \quad \Gamma \vdash x : A}{\Gamma \vdash f\, x : B},$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B} \qquad \frac{\Gamma \vdash s : A + B \quad A :: \Gamma \vdash l : C \quad B :: \Gamma \vdash r : C}{\Gamma \vdash \mathsf{case}\ s\ l\ r : C}.$$

The current term language's constructor tags and projections can be represented as typed sugar. Proving the desugaring once keeps the kernel of the formalization small.

6.3.3 Unification obligations

The current checker uses proof-type metavariables because terms such as empty case analysis can inhabit an arbitrary expected type and because some constructor forms infer only partial information. A formal unifier needs these lemmas:

1. **Substitution lookup soundness:** lookup and application agree on mapped variables.
2. **Occurs-check acyclicity:** inserting $\alpha \mapsto \tau$ when $\alpha \notin \mathrm{FV}(\tau)$ preserves finite expansion.
3. **Unification preservation:** if the accumulated substitution satisfies prior constraints, one successful step preserves that invariant.
4. **Success theorem:** a successful run produces a substitution satisfying every input equation.
5. **Typing reconstruction:** inferred proof types and constraints imply declarative typing after substitution.

A most-general-unifier theorem is unnecessary for soundness. It is useful only if checker completeness or principal typing is later desired.

6.3.4 Relating the Lean checker to Haskell certificates

The serialization format should contain only canonical data:

- a version number;
- normalized formula syntax with atom identifiers;
- normalized term syntax with de Bruijn indices;
- constructor arities and branch metadata;
- optionally, a hash of the original Djinn formula and environment.

The Lean parser is part of the checker. Haskell's pretty-printer should not be trusted as the only identity boundary. A round-trip theorem for the canonical format is desirable; a parser failure simply rejects the certificate.

6.4 Formalizing Exference's candidate checker

Exference's checker is richer than Djinn's because it includes local/global bindings, rigid skolems, polymorphic schemes, deconstructors, class constraints, and residual constraint solving. The recommended proof decomposition is:

1. Formalize the shared type syntax and capture-avoiding substitution.
2. Prove the relevant unifier sound, separately for flexible and rigid variables.
3. Define declarative expression typing with explicit environments.
4. Prove each expression form in the executable checker preserves the typing invariant.
5. Prove class/instance resolution sound relative to a declarative entailment relation.
6. Prove that a successful top-level check has no escaping rigid variables and no unsolved required constraints.

This theorem certifies candidate validity but makes no claim that search found all candidates. That separation matches the engine's intended heuristic nature.

6.5 Ranking and simplification are secondary theorems

The order in which candidates are shown, alpha-renaming choices, and simplification quality are observable behavior but not logical soundness. They should be specified only after the checker boundary is certified.

Useful later theorems include:

- simplification preserves typing;
- simplification preserves denotation in a chosen total semantics;
- alpha-renaming preserves scope and typing;
- candidate score sorting is stable and respects the documented ordering;
- bounded enumeration produces a prefix of a larger run under the same deterministic strategy.

The current property tests already exercise some of these claims. They should remain as cross-language refinement tests even after formal proofs exist.

Chapter 7

Certifying negative results

7.1 Why search exhaustion is not enough

A negative result must close two independent gaps:

1. **Calculus gap:** show that the abstract sequent has no derivation.
2. **Abstraction gap:** show that every permitted source inhabitant would yield such a derivation.

The first can be certified without trusting the search implementation. The second is Leant-specific and controls the scope of the claim.

7.2 Option A: a complete refutation DAG

The most direct certificate mirrors backward proof search. Each node contains a canonical sequent and either:

- a rule application whose children are all required premises;
- a locally closed failure reason, such as an atomic succedent absent from an irreducible context;
- references to earlier nodes, allowing DAG sharing.

The checker validates:

1. the selected rule is applicable;
2. every required branch is present;
3. each child sequent is exactly the rule's premise after normalization;
4. leaves satisfy a proved failure condition;
5. references are acyclic or decrease under a certified measure.

The soundness theorem is proved by induction over the certificate: if a derivation of the parent existed, invertibility or completeness of the selected rule would produce a derivation in at least one child, contradicting the children's certificates.

This approach is close to Djinn's existing search and can preserve diagnostics. Its disadvantages are potentially large certificates and the need to formalize the exact completeness/invertibility facts for every branching rule.

7.3 Option B: a finite Kripke countermodel

For the pure intuitionistic propositional fragment, a finite Kripke countermodel is often a better negative certificate. The producer supplies:

- a finite set of worlds;
- a decidable partial order representing information growth;
- a monotone valuation of atoms;
- a designated root world;
- evidence, checked by computation, that the goal formula is not forced at the root while all assumptions are forced there.

  Lean defines forcing recursively:

$$\begin{aligned} w \Vdash p &\iff V(w,p), \\ w \Vdash A \wedge B &\iff w \Vdash A \ \wedge \ w \Vdash B, \\ w \Vdash A \vee B &\iff w \Vdash A \ \vee \ w \Vdash B, \\ w \Vdash A \to B &\iff \forall v \geq w,\ v \Vdash A \to v \Vdash B, \\ w &\nVdash \bot. \end{aligned}$$

  The one substantial metatheorem is standard semantic soundness:

$$\Gamma \vdash A \quad \Longrightarrow \quad \forall M, w,\ w \Vdash \Gamma \to w \Vdash A.$$

A checked countermodel therefore proves non-derivability. The untrusted producer does not need a completeness proof and may use any model-finding strategy.

7.3.1 Why this is particularly attractive

A countermodel certificate has several advantages:

- It avoids proving that the exact Haskell search exhausted every branch.
- It is independent of proof-term generation and can catch common-mode bugs.
- It gives a human-readable explanation: a small information-growth scenario in which the formula fails.
- It handles classically valid but intuitionistically invalid formulas, such as excluded middle, by using multiple worlds rather than an ordinary truth table.
- It can be generated after Djinn reports exhaustion; failure to find a model merely downgrades the result instead of threatening soundness.

The finite-model property guarantees that invalid IPC formulas have finite Kripke countermodels, but the certificate architecture needs only semantic soundness, not a formal proof of the generator's completeness [11, 12].

7.3.2 Recommended negative certificate strategy

Implement both formats behind one interface:

Listing 7.1: Two independently checkable negative certificate forms.

```
inductive NegativeCertificate (s : Sequent) : Type
  | refutationDAG (c : RefutationDAG s)
  | kripkeModel  (m : Countermodel s)
```

The Kripke route should be the default for pure IPC. The refutation DAG remains useful for formulas with implementation-specific nominal metadata, for proof-search debugging, and as a route to a full LJT formalization.

7.4 The exact formal statement of "refuted"

For the first certified fragment, the result should be:

$$\mathsf{CertifiedRefutation}(\Gamma, A) :\equiv \neg\mathsf{Core.Derives}(\Gamma, A).$$

Leant may lift this through an abstraction certificate:

$$\mathsf{CertifiedRefutation}(\Gamma, A) \wedge \mathsf{Abstracts}(E, T, L, \Gamma, A) \Longrightarrow \neg\mathsf{SourceTerm}_L(E, T).$$

Notice what is absent: no claim that $T \to \mathsf{False}$ is inhabited, no claim about all future environment extensions, and no claim about terms using disallowed constants.

7.5 Provider and environment closure

A source term may mention a global declaration. To obtain a full-environment theorem, every such declaration must be covered by the abstraction. There are three defensible policies.

7.5.1 Policy 1: pure structural terms

Forbid global constants except constructors and eliminators used to implement the recognized structural connectives. This produces the cleanest theorem and exactly matches traditional type-directed "plumbing" synthesis.

The result is labeled:

> No inhabitant exists in the pure structural term language.

7.5.2 Policy 2: explicit certified provider set

Allow a finite set P of constants. Each provider carries an elaborated Lean type and a checked abstraction to a core premise. The negative result is relative to P:

$$\neg\mathsf{SourceTermUsing}(E, P, T).$$

This is well suited to Leant's provider lanes and library ratings. The provider set must be displayed or stored in the certificate manifest.

7.5.3 Policy 3: closed environment snapshot

Model every usable constant in an environment snapshot and prove that source terms cannot refer outside it. This is the strongest policy but also the most expensive. Definitions may need unfolding summaries, opaque constants become assumptions, recursive constants require a term-language policy, and unsafe or axiom-bearing declarations must be classified.

A bounded provider inventory is not evidence of closure. It can improve positive search while negative certification remains relative to the structural language or explicit set.

7.6 Approximation direction matters

The translator may intentionally abstract source structure. For negative soundness:

- **Merging distinct atoms** is generally safe as an over-approximation. If a source derivation exists, uniformly substituting one atom for several preserves derivability; therefore underivability after merging implies underivability before merging.
- **Splitting equal atoms** is unsafe. It may destroy a proof such as identity and create a false refutation. Canonical identity must never assign different abstract atoms to definitionally equal source expressions.
- **Treating a structured proposition as opaque** is usually unsafe. Removing introduction and elimination structure can make the abstract problem harder, leading to false negatives.
- **Adding extra premises or constructors** is safe for a negative over-approximation, although it may destroy completeness of refutation and reduce the number of certified negatives.
- **Dropping premises or constructors** is unsafe unless accompanied by a theorem showing that they are irrelevant to every permitted source term.

This gives a formal interpretation to Leant's current unsafe-atom and depth checks. The proof-carrying replacement should record the direction of each abstraction step.

7.7 A practical negative certificate protocol

A versioned certificate envelope should contain:

Field	Purpose
Schema version	Reject incompatible syntax or rule changes deterministically.
Engine/revision metadata	Diagnostic only; not trusted for soundness.
Canonical sequent	The exact formula and premise environment checked by Lean.
Atom table	Stable canonical identities and optional source pretty names.
Projection manifest	Target/environment fingerprint, scope policy, provider set, and abstraction evidence identifiers.
Negative payload	Refutation DAG or finite Kripke model.
Digest	Detect accidental mismatch between Haskell output, Lean check, and displayed report.
Optional trace	Human-readable search/model explanation, not trusted.

The checker returns a theorem object only after parsing and validating every trusted field. Unknown optional metadata is ignored; unknown trusted fields or versions cause rejection.

Chapter 8

Formalizing Djinn and its LJT engine

8.1 The implementation-specific target

It would be a mistake to prove only the textbook statement "G4ip is sound and complete" and then declare `Djinn/Internal/LJT.hs` verified. The current implementation adds engineering structure that must be connected to the calculus:

- n-ary products and constructor-tagged disjunctions;
- nominal empty formulas;
- context indexes and reachability pruning;
- proof-term construction interleaved with search;
- explicit alternatives, depth-first or interleaved scheduling, and choice-point budgets;
- special rules for antecedent implications and negations;
- deletion or transformation of formulas to avoid repeated contraction;
- conversion between Haskell type structure and LJT formulae.

The formal project should therefore define an abstract calculus first and then prove that each implementation layer refines it.

8.2 Suggested Lean module layout

A practical package layout is:

Listing 8.1: Suggested formal package layout.

```
DjexCert/
  Formula.lean          -- atoms, products, sums, implication, bottom/top
  Term.lean             -- de Bruijn proof terms
  Typing.lean           -- declarative natural-deduction typing
  Normalize.lean        -- n-ary/binary normalization and metadata checks
  Unify.lean            -- checker metavariables and substitutions
  Check.lean            -- executable proof checker
  CheckSound.lean       -- check_sound theorem
  Sequent.lean           -- G4ip/LJT sequents and derivations
  Rules.lean             -- rule soundness/invertibility
  Measure.lean           -- termination ordering
```

```
  Decide.lean            -- verified decision procedure
  SearchCert.lean        -- refutation DAG checker
  Kripke.lean            -- semantics and countermodel checker
  Codec.lean             -- canonical certificate format
  Examples.lean          -- regression corpus
```

This isolates the minimal trusted checker from optional proofs about the producer and search order.

8.3 Formula normalization and metadata

The implementation's `Conj [Formula]` and constructor-tagged `Disj [(ConsDesc, Formula)]` are convenient for source reconstruction. There are two reasonable formal representations.

8.3.1 Direct n-ary representation

Keep vectors of fields and branches, with well-formedness predicates for constructor names and arities. Typing rules quantify over vector indices. This makes certificate comparison close to Haskell but increases dependent-vector bookkeeping.

8.3.2 Binary logical core plus annotations

Normalize products and sums to a binary tree and keep constructor metadata in a separate annotation proved consistent with the logical tree. This gives much simpler metatheory. Rendering can consume the annotation; proof checking and Kripke semantics consume the binary core.

The second design is recommended. The normalization theorem should state both directions of derivability for well-formed formulas:

$$\mathsf{Derives}_{n\text{-ary}}(\Gamma, A) \iff \mathsf{Derives}_{binary}(N\Gamma, NA).$$

8.4 Nominal empty types

The implementation distinguishes empty types by symbol. Logically, every genuinely empty inductive type supports elimination to any target, so each behaves like bottom for inhabitation. The formalization can either map all nominal empties to $\bot$ or retain labels and give each the same elimination rule.

Retaining labels is useful for source reconstruction and diagnostics. The logical theorem can then prove that erasing labels preserves derivability. What must not happen is to treat a merely opaque atomic type as empty; the Leant projection must provide evidence that the source head has no constructors in the advertised environment.

8.5 Natural deduction as the checker specification

Use a typed de Bruijn syntax such as:

Listing 8.2: Illustrative core syntax; exact names may change.

```
namespace DjexCert

inductive Ty where
  | atom   : Nat -> Ty
  | zero   : Ty
  | one    : Ty
  | prod   : Ty -> Ty -> Ty
```

```
  | sum    : Ty -> Ty -> Ty
  | arr    : Ty -> Ty -> Ty
  deriving DecidableEq, Repr

inductive Tm where
  | var    : Nat -> Tm
  | lam    : Tm -> Tm
  | app    : Tm -> Tm -> Tm
  | unit   : Tm
  | pair   : Tm -> Tm -> Tm
  | fst    : Tm -> Tm
  | snd    : Tm -> Tm
  | inl    : Tm -> Tm
  | inr    : Tm -> Tm
  | caseOf : Tm -> Tm -> Tm -> Tm
  | absurd : Tm -> Tm
  deriving DecidableEq, Repr

inductive HasType : List Ty -> Tm -> Ty -> Prop where
  | var    : ctx[n]? = some a -> HasType ctx (.var n) a
  | lam    : HasType (a :: ctx) body b ->
             HasType ctx (.lam body) (.arr a b)
  | app    : HasType ctx f (.arr a b) -> HasType ctx x a ->
             HasType ctx (.app f x) b
  | unit   : HasType ctx .unit .one
  | pair   : HasType ctx x a -> HasType ctx y b ->
             HasType ctx (.pair x y) (.prod a b)
  | inl    : HasType ctx x a -> HasType ctx (.inl x) (.sum a b)
  | inr    : HasType ctx y b -> HasType ctx (.inr y) (.sum a b)
  | caseOf : HasType ctx s (.sum a b) ->
             HasType (a :: ctx) l c ->
             HasType (b :: ctx) r c ->
             HasType ctx (.caseOf s l r) c
  | absurd : HasType ctx x .zero -> HasType ctx (.absurd x) a

end DjexCert
```

This is an interface sketch rather than a claim that the listing was compiled verbatim. The production formalization should use library lookup lemmas and explicit weakening/lifting operations.

8.6 Proof checker proof plan

8.6.1 Phase 1: a bidirectional checker without metavariables

Start with an expected-type-directed checker:

- lambdas are checked against arrows;
- pairs against products;
- injections against sums with the expected opposite branch known;
- case branches against a common expected type;
- applications infer the function's domain/codomain.

This checker is easier to prove and can validate a canonical certificate form in which ambiguous constructs carry type annotations.

8.6.2 Phase 2: compatibility with the current Haskell term format

Add proof-type metavariables and unification so the Lean checker accepts certificates close to the existing `ProofCheck.hs`. Prove a translation from the annotated canonical term to the Haskell-style term and, optionally, completeness of elaboration.

This two-stage route reduces risk: the trusted kernel can remain the simple annotated checker even if Haskell emits a compact term plus an untrusted elaboration hint.

8.6.3 Phase 3: Haskell refinement tests

Generate formulas and proof terms in Haskell; serialize them; and require agreement among:

- the existing Haskell checker;
- the Lean executable checker;
- the Lean declarative theorem obtained from successful checking.

Mutation tests should alter constructor tags, branch arities, variable indices, and expected types to ensure certificates are rejected.

8.7 Formalizing the LJT rules

8.7.1 Separate derivability from the algorithm state

Define a clean sequent calculus `LJT.Derives` whose contexts are multisets or normalized lists. Then define the executable state used by search, with indexes and caches. Prove a representation invariant connecting them.

This separation prevents performance structures from contaminating every logical proof. Typical invariants include:

- each indexed formula occurs in the logical context;
- no required formula is lost by normalization;
- index categories partition formulas according to their outer connective;
- cached reachability information is conservative;
- fresh proof-term variables correspond bijectively to context assumptions.

8.7.2 Rule soundness

For every backward rule, prove that derivations of all premises yield a derivation of the conclusion. For proof-producing search, strengthen the theorem to construct the corresponding natural-deduction term.

The implication-left cases are the delicate part. G4ip/LJT replaces unrestricted contraction with specialized rules based on the antecedent's shape. The proof should follow the corrected calculus rather than infer correctness from operational intuition.

8.7.3 Invertibility and completeness

A decision procedure needs the converse direction appropriate to each deterministic reduction: if the conclusion is derivable, then the selected premises contain enough information to continue. Branching rules require at least one branch to be derivable; conjunctive proof obligations require all children.

The implementation's rule priority must be covered. A theorem about a nondeterministic calculus does not automatically justify a deterministic rule-selection strategy unless the selected rules are invertible or permutations are proved admissible.

8.7.4 Termination

Dyckhoff's 1992 paper gives the classic contraction-free calculus, but its termination argument should be formalized using the shortened 2018 correction [8, 9, 10]. The implementation proof should define a well-founded measure over sequents and show strict decrease for every recursive call after normalization.

Candidate measures include a multiset of formula weights refined by implication shape. The formalization should avoid an opaque `termination_by size` proof that accidentally relies on implementation artifacts; a named mathematical measure will be reusable for the certificate checker and documentation.

8.8 Pruning and reachability

The implementation contains a `goalMayBeReachable` optimization. Such pruning is sound only if

$$\neg\mathsf{goalMayBeReachable}(s) \Longrightarrow \neg\mathsf{Derives}(s).$$

This theorem may be easier to prove semantically: construct a simple valuation or syntactic invariant showing that no antecedent can produce the goal's head atom. If the current predicate is too implementation-entangled, the formal checker should ignore the pruning decision. The producer can prune for speed, but a negative certificate must independently close the branch.

This illustrates the general architecture: an optimization need not be verified to remain in production; it only needs to stop being trusted as evidence.

8.9 Budgets, laziness, and fairness

The Haskell engine uses a lazy search structure with configurable alternatives, scheduling, and optional choice-point budget. Formalizing the exact lazy stream is unnecessary for the first logical theorem.

Use three layers:

1. a total decision procedure returning one derivation or a refutation;
2. a finite bounded evaluator with a status theorem;
3. an enumeration relation describing the set of proof terms the lazy Haskell search may emit.

Then prove:

- every enumerated term checks;
- budget exhaustion returns only a bounded status;
- unbounded decision is complete and terminating;
- optional fairness/order properties separately, if they matter to user-visible ranking.

This avoids trying to prove productivity and exact interleaving before the core logic is secure.

8.10 Self-reference and recursive targets

Djinn distinguishes full uninhabitability from cases where every candidate requires a distinguished target reference. This is not ordinary IPC non-derivability; it is a provenance-sensitive statement.

Formalize it by extending the context with a marked assumption $r : A$ and defining whether a derivation uses r. The theorem can state:

$$\forall d : \Gamma, r : A \vdash A,\ \mathsf{uses}(d, r).$$

A certificate may annotate each derivation or branch with a dependency bit. This turns the current operational distinction into a precise theorem and may also support cycle diagnostics for recursive provider synthesis.

8.11 Do we need a full LJT proof if countermodels are checked?

Not for external negative soundness. A checked Kripke countermodel already proves non-derivability. A complete LJT formalization remains valuable for four reasons:

- it verifies the engine's decision and termination claims;
- it can generate proof terms and refutation certificates inside Lean;
- it validates search optimizations and status reporting;
- it provides a reference implementation against which Haskell can be differentially tested.

The certificate architecture lets this work proceed without blocking a trustworthy product boundary.

Chapter 9

Formalizing Exference without overclaiming

9.1 The correct target is partial correctness

Exference is intentionally heuristic. Its formal contract should be:

1. every emitted candidate is well typed in the declared source calculus;
2. every transformation between checked boundaries preserves typing or is rechecked;
3. every operational status accurately describes why enumeration stopped;
4. configured finite runs terminate;
5. no negative logical conclusion is inferred from a bounded or pruned run.

Completeness is not required and should not be smuggled in through the word `Exhausted`. Exhaustion is always relative to the configured state graph and pruning policy.

9.2 Shared type formalization

The shared type syntax includes variables, constructors, application, arrows, tuples, and explicit forall types with constraints. A robust Lean representation should distinguish:

- flexible unification variables;
- rigid skolems;
- bound type variables, preferably by de Bruijn index;
- globally named type constructors;
- quantified schemes and their binder scope;
- class constraints as nominal predicates over types.

The main foundational theorems are:

- substitution is capture-avoiding;
- opening and closing binders are inverse under freshness conditions;
- rigid variables are never solved by flexible substitution;

- alpha-equivalent schemes have equal checking behavior;
- well-kindedness is preserved by substitution.

Even if the first checker proof erases kinds, kind soundness should be added before claiming source-Haskell correctness.

9.3 Unification and constraint solving

Exference has several related unifiers with offsets and directional restrictions. They should not be forced into one generic implementation merely because their code resembles one another. Formalize a common declarative relation, then prove each executable variant satisfies the relation it needs.

For class constraints, define an entailment relation generated by:

- assumed query constraints;
- superclass closure;
- matching instance heads;
- recursively discharged instance prerequisites;
- cycle guards with a stated semantics.

Soundness means that every solved constraint has a derivation in this relation. Completeness may fail due ordering, overlap policy, and cycle cutoffs; that is acceptable if residual constraints are not silently dropped.

9.4 Search-control theorem

Model the scheduler as a finite transition system parameterized by limits. The status theorem can be stated as:

Listing 9.1: Operational status honesty.

```
theorem run_status_sound :
  match run cfg initial with
  | .exhausted out => noRunnableState cfg out.finalState
  | .stepLimit out => out.steps = cfg.maxSteps
  | .pruned out    => out.prunedBranches > 0
  | .idExhausted out => noFreshIdentifier out.finalState
  | .running _     => True
```

The exact data structures may differ, but each status must have a mechanically checkable witness. A status should never be inferred from an empty lazy list whose evaluation may have been truncated elsewhere.

9.5 Simplification and rendering

Historical audits in the repository found that checked candidates could once be changed by simplification or captured by rendering. The current architecture has been hardened, but these incidents identify excellent regression theorems:

- simplification does not introduce a global name absent from the allowed environment;

- generated binder spellings are injective and disjoint from rendered global spellings;
- qualification policy preserves global identity;
- rendering followed by parsing/elaboration preserves the intended binding graph;
- any transformation after checking is either proved type-preserving or followed by rechecking.

Leant's final Lean gate already contains the damage from renderer mistakes. Formalizing the properties is still valuable for completeness, deterministic output, and direct Haskell use outside Leant.

9.6 What not to prove

Do not state that Exference is complete for rank-N Haskell inhabitation. Its bounds, pruning, rating, omission policy, class solver, and finite identifier supply are deliberate. The meaningful theorem is that success is valid and failure is honestly labeled.

Chapter 10

Formalizing Leant's projection

10.1 The serializer is already in the right language, but not yet proof carrying

Leant's serializer is generated Lean meta code executed over an elaborated `Lean.Expr`. It instantiates metavariables, reduces to weak-head normal form, recognizes structural constants and inductive families, and emits an S-expression consumed by Haskell. This is a much better starting point than a textual parser because the source object is already elaborated and name-resolved.

However, Lean meta code is not trusted merely because it is written in Lean. `MetaM` can inspect and construct expressions incorrectly, pretty-printer strings are not canonical identities, and the serializer currently emits data plus boolean-style completeness information rather than a theorem. The proposed change is:

> The serializer becomes an untrusted *certificate producer*; a small Lean checker validates a typed projection certificate against the original target expression.

10.2 Replace Frag with a checked abstraction object

The current `Frag` datatype usefully records arrows, products, sums, top, bottom, quantified binders, exact contexts, atoms, applications, and finite or recursive inductive descriptions. The formal checker should split this into:

1. a small logical formula consumed by the core checker;
2. a source-node description retaining the exact Lean expression, sort, head constant, parameters, and constructor/eliminator evidence;
3. an abstraction derivation showing how the source node maps to the logical formula.

An illustrative evidence family is:

Listing 10.1: Projection evidence should explain each abstraction step.

```
inductive Abstracts (env : LeanEnv) : LeanExpr -> Core.Ty -> Prop
  | atomMerge
      (hCanonical : canonicalKey e = k)
      (hSafe : PureOpaqueAtom env e) :
      Abstracts env e (.atom k)
  | arrow
      (ha : Abstracts env a A)
```

```
    (hb : Abstracts env b B) :
    Abstracts env (mkArrow a b) (.arr A B)
| product
    (headEvidence : ProductHead env head a b)
    (ha : Abstracts env a A)
    (hb : Abstracts env b B) :
    Abstracts env (mkApp2 head a b) (.prod A B)
| sum
    (headEvidence : SumHead env head a b)
    (ha : Abstracts env a A)
    (hb : Abstracts env b B) :
    Abstracts env (mkApp2 head a b) (.sum A B)
| finiteInductive
    (schema : CheckedFiniteSchema env e core) :
    Abstracts env e core
| overApprox
    (h : EverySourceTermMaps env e core) :
    Abstracts env e core
```

The constructor names are schematic. The important requirement is that a certificate cannot claim negative safety without supplying the corresponding proof object.

10.3 Recognized connectives

The current serializer recognizes multiple Lean encodings of the same logical shape: `And`/`Prod`/`PProd`, `Or`/`Sum`/`PSum`, `True`/`Unit`/`PUnit`, `False`/`Empty`/`PEmpty`, `Iff`, and `Not`. The formalization should prove a small lemma for each recognized head.

10.3.1 Products

For `And`, `Prod`, and `PProd`, prove that any source inhabitant yields two component inhabitants and that component inhabitants reconstruct a source inhabitant. Only the first direction is required for negative soundness. The exact sort and universe levels must be retained in source evidence even if the logical core erases them.

10.3.2 Sums

For `Or`, `Sum`, and `PSum`, prove that any source inhabitant is produced by one of the recognized constructors and therefore maps to a core injection. This uses the inductive declaration's constructor metadata, not only the printed head name.

10.3.3 Top and bottom

For unit-like heads, map the unique constructor to top. For empty-like heads, validate from the environment that the inductive declaration has no constructors and supports the expected elimination. A merely unknown constant with an evocative name must not be accepted as bottom.

10.3.4 Iff and Not

`Iff p q` is structurally a pair of implications, while `Not p` unfolds to `p -> False`. The certificate checker can either reduce these definitionally or use head-specific abstraction lemmas. Keeping the exact source head in evidence improves diagnostics and replay stability.

10.4 Sort and universe information

The logical core deliberately forgets whether a product came from `And : Prop` or `Prod : Type`. This is acceptable only because the projection evidence records a source-specific mapping. A future direct renderer theorem cannot reconstruct Lean syntax from the erased formula alone.

For negative adequacy, the main issue is not universe equality but whether every source introduction/elimination form maps to the core connective. The checker should nevertheless retain:

- the sort of each source node;
- universe levels and level parameters;
- explicit versus implicit binder information;
- dependency information for forall binders;
- the exact head constant after weak-head reduction.

This prevents a proof for one head or universe instance from being reused accidentally for another.

10.5 Atom identity: canonicalization is a proof obligation

The current serializer uses printed or generated keys to identify opaque atoms and records whether an atom is safe based partly on whether the expression uses constants. For positive search, a bad key is harmless because Lean rejects invalid reconstructions. For negative search, key behavior is logical.

There are two failure modes:

False splitting
: Two definitionally equal source expressions receive different abstract atoms. This can remove an identity proof and create a false negative. It is unacceptable.

False merging
: Two distinct source expressions receive one abstract atom. This usually makes the abstract calculus more permissive. It may create extra positive candidates, but underivability in the merged abstraction remains safe by uniform substitution. It is therefore an acceptable deliberate over-approximation when documented.

The certificate checker should avoid relying on pretty printing. Better canonical identities are:

- locally nameless expressions after metavariable instantiation and controlled reduction;
- structural hashes checked by structural equality rather than trusted alone;
- equivalence classes generated by a verified union operation that only merges, never splits, occurrences known to be the same source node;
- explicit source-expression references within one certificate rather than global strings.

A theorem should state that every repeated source occurrence maps consistently. Deliberate merging can be represented as a quotient map from source atom IDs to core atom IDs.

10.6 Opaque global constants

The current notion of an unsafe atom correctly reflects a major danger: replacing a structured global proposition or type by an opaque atom can remove constructors, eliminators, reducibility, or existing inhabitants. This is an under-approximation and cannot support a full negative claim.

There are three safe responses:

1. Refuse negative certification and continue positive search only.
2. Add an abstract premise representing every known global inhabitant/provider, producing an over-approximation.
3. Supply a head-specific abstraction theorem or an exact finite inductive schema.

The first should remain the default for unknown constants.

10.7 Non-recursive inductive families

A non-recursive inductive instance can be abstracted as a finite sum of constructor payload products. A checked schema must validate:

- the exact inductive head and parameters;
- every constructor of the instantiated family is included;
- constructor result indices unify with the target instance;
- constructor arguments are classified correctly as parameters, indices, recursive occurrences, and payload fields;
- inaccessible or dependent fields are handled conservatively;
- no constructor is silently discarded by a depth or representation bound.

For a simple algebraic datatype

$$D\ \vec{p} = C_1\ \vec{A}_1 \mid \cdots \mid C_n\ \vec{A}_n,$$

the negative-sound abstraction theorem maps a hypothetical source value by induction/case analysis to one of the core branches. The converse reconstruction theorem is optional because final Lean elaboration checks positive candidates.

Indexed families require more care: some constructors may be impossible at a particular index. Including impossible constructors is a safe over-approximation for negative results; excluding a possible constructor is unsafe. Therefore the checker may choose a simple policy that includes all constructors whose impossibility is not formally established.

10.8 Recursive inductive families

The current code preserves bounded one-layer elimination for compatible recursive families and marks recursive completeness separately. This is an excellent positive feature but not an immediate basis for a full negative theorem.

A one-layer abstraction that replaces recursive tails with opaque atoms is generally an under-approximation if recursive elimination or induction could construct the target. Safe options are:

- positive-only treatment, with negative evidence disabled;
- a coarser over-approximation that adds enough constructors/eliminators to include every source use, accepting that few formulas will refute;
- a formal polynomial-functor/fixed-point semantics for a restricted strictly positive family;

- a normal-form theorem showing that every inhabitant relevant to the target uses only the bounded eliminations represented.

The first is the correct initial policy. Recursive-family negative certification should be a later theorem family, not a boolean extension of the finite case.

10.9 Forall, rank-N types, and parametricity

Leant and Djex support sophisticated bounded instantiation plans for quantified hypotheses and impredicative targets. These plans are highly useful for positive synthesis because final Lean checking filters invalid choices. They are much harder to justify for negative results.

Treating a polymorphic type variable as an opaque propositional atom relies on a parametricity principle: source terms must not inspect or branch on the identity of the type. In Haskell, bottoms, `seq`, type classes, `Typeable`, unsafe coercions, and non-parametric primitives complicate this statement. In Lean, dependent elimination, type classes, universe polymorphism, quotients, and arbitrary global constants complicate it differently.

A certified negative rank-N fragment needs one of the following; the logical-relations route is the classic parametricity approach [13]:

1. a syntactically restricted System F-like source term language with a proved abstraction theorem;
2. a logical-relations/parametricity theorem for the admitted Lean fragment;
3. explicit semantic counterexamples specialized to the closed target;
4. a conservative over-approximation that includes all bounded instantiations and refuses to refute when the bound is not proved complete.

The fourth matches the current honesty policy and should remain in place. Bounded instantiation can contribute candidates; it cannot contribute general negative evidence without a completeness theorem for the frontier.

10.10 Type classes and instances

Contexts and instances become premises or provider-like operations in the abstract calculus. A sound negative theorem requires a closed account of available instance resolution. Important questions include:

- Are superclass projections included?
- Are overlapping instances resolved with the same priority as Lean?
- Can local instances or synthesized instances appear outside the inventory?
- Are instance methods opaque providers, or are their definitions unfolded?
- Can type-class search use recursive or generated instances not represented in the core?

Until these are formalized, constraints should either disable negative evidence or be represented by an explicit finite assumption set. Positive terms remain safe through final Lean checking.

10.11 Local hypotheses and providers

Local hypotheses are easier than global providers because the target context explicitly lists them. The projection certificate can map each accessible local hypothesis to a core premise and prove that source use of the hypothesis corresponds to the core variable rule.

Quantified local providers need an instantiation policy. For negative soundness, the abstract environment must include every instantiation that a permitted source term could make, or use a genuinely quantified core calculus. A finite query-derived frontier is not complete without a separate theorem. Therefore quantified-provider lanes should remain positive-only unless the source term language itself is restricted to that frontier.

10.12 Depth limits and partial translation

The current translator uses explicit fuel and emits a depth marker when it stops. This is operationally honest. The formal checker should interpret a depth marker as a refusal to establish negative adequacy, not as an atom.

A useful evidence type is:

Listing 10.2: Projection result with explicit proof strength.

```
inductive ProjectionResult (target : LeanExpr) : Type
  | certified (core : Core.Ty)
      (scope : Scope)
      (adequate : NegativeAdequate target scope core)
  | positiveOnly (core : Core.Ty)
      (reasons : Array ApproximationReason)
  | refused (reason : RefusalReason)
```

The Haskell engine can search both `certified` and `positiveOnly` results. Only the first can combine with a negative certificate.

10.13 Proving the projection checker, not the MetaM serializer

The serializer may remain complex and partial. It emits:

- references to source expressions;
- proposed connective classifications;
- constructor schemas;
- atom quotient maps;
- provider mappings;
- scope declarations.

A pure or mostly pure Lean checker validates these claims by querying the pinned environment through a narrow API and constructs the abstraction proof. This is analogous to elaboration followed by kernel checking: the producer is convenient; the checker is authoritative.

Chapter 11

Can Leant prove that no Lean term exists?

11.1 Three different domains of quantification

The sentence "no Lean term of type T exists" can quantify over:

1. source terms in a small structural DSL;
2. elaborated kernel expressions using a finite allowed constant set;
3. all kernel expressions typeable in the entire current environment.

The first is readily certifiable. The second is feasible for carefully closed environments and fragments. The third is dramatically stronger and usually inappropriate for an interactive synthesizer with bounded provider discovery.

11.2 Why the ambient environment defeats an unrestricted claim

Suppose the pure structural calculus proves T underivable. The live Lean environment may nevertheless contain:

- a declaration whose type is definitionally equal to T;
- an axiom such as a polymorphic inhabitant;
- a theorem deriving T from imported axioms;
- a recursive or opaque definition whose type is T;
- a provider hidden by namespace, rating, inventory cap, or query relevance filter.

No amount of completeness in LJT excludes these terms unless the environment abstraction includes them. A formal theorem forces this scope question into the type of the result, which is exactly where it belongs.

11.3 A restricted kernel-term theorem

A meaningful intermediate target is to define a reified subset of Lean expressions:

- variables, lambdas, and applications;

- recognized product/sum/unit/empty constructors and eliminators;
- a finite certified constant set;
- no recursion, unsafe declarations, macros, metavariables, or sorry;
- controlled universe and implicit-argument forms.

Then prove a reification theorem:

$$E \vdash e : T \ \wedge \ \mathsf{InRestrictedGrammar}(e, L) \Longrightarrow \mathsf{Derives}(\alpha(E, L, T)).$$

This produces a genuine statement about elaborated Lean terms, not merely generated syntax, while keeping normalization and environment closure manageable.

11.4 Using lean4lean infrastructure

The `lean4lean` repository contains definitions and verification work for Lean expressions, levels, environments, local contexts, type checking, and typing metatheory. It may substantially reduce the cost of stating a restricted-kernel theorem. Potential reuse points are:

- reified expression and environment formats;
- declarative typing relations;
- verified type-checker components;
- environment boundary and replay machinery;
- expression traversal and name handling.

The new Djex/Leant formalization should not depend on the whole project initially. A small adapter can import only stable expression/typing definitions. Domain-specific structural adequacy remains a separate theorem.

11.5 Kernel independence and revision pinning

On 28 July 2026, Lean issue #14576 demonstrated that the checked declaration path in a then-current nightly accepted a malformed nested-inductive projection and permitted an axiom-free proof of `False`. Pull request #14577 added the missing type check for parametric arguments of nested inductives and was merged the same day [14, 15]. The Lean revision inspected for this report is after that fix.

The lesson is not that kernel checking is useless; it is that evidence should be reproducible, revision-pinned, and preferably checked through structurally diverse implementations. Discussion around the fix also showed the limits of common-mode diversity: a checker derived closely from the official implementation may reproduce the same omission, whereas a checker organized around independently rechecking exported declaration signatures can reject it.

For Leant artifacts, record:

- exact Lean commit/toolchain;
- exact imported module hashes;
- whether kernel checking was enabled;
- checker implementations used;
- dependency/axiom manifest;

- certificate checker revision.

A small domain-specific proof certificate remains the most attractive independent layer because its checker can be orders of magnitude smaller than a complete Lean kernel.

11.6 Object-level refutations as an optional stronger result

When possible, Leant can synthesize a Lean term of $T \to \mathsf{False}$, or a theorem $\neg\mathsf{Nonempty}\,T$, and kernel-check it. This is stronger and easier to consume than a meta-level derivability certificate.

A negative-result pipeline may therefore try, in order:

1. synthesize and kernel-check an object-level refutation;
2. check a semantic/core non-derivability certificate and report its exact scope;
3. otherwise report only bounded failure.

The first route is not complete in intuitionistic logic, so the second remains essential.

Chapter 12

Recommended certification architecture

12.1 Design goal

Keep Djex and Leant fast and feature-rich while shrinking the trusted computing base. Search, ranking, provider discovery, rendering heuristics, and certificate generation may remain in Haskell and Lean meta code. A small Lean library validates canonical artifacts and exposes theorems.

12.2 Components

12.2.1 DjexCert: the logical core

`DjexCert` is a standalone Lean package with no dependency on Leant or the Lean metaprogramming framework. It contains:

- core formulas and terms;
- well-formedness and declarative typing;
- executable positive checker and soundness theorem;
- Kripke semantics and countermodel checker;
- optional LJT decision procedure and refutation-DAG checker;
- canonical certificate codec;
- extraction-friendly executable entry points.

This package should be small enough to audit independently and stable enough to version conservatively.

12.2.2 Djex producer adapters

The Haskell repository gains producer-only modules:

- convert the current formula/term representation to canonical certificates;
- request a finite countermodel after a complete Djinn miss;
- include precise bound and approximation metadata;
- never label a result certified until the Lean checker returns success.

A local checker executable can be invoked by pipe, file, or FFI. A failed checker call degrades the result to diagnostic output; it does not crash the REPL or produce a trusted claim.

12.2.3 LeantProjectionCert: environment-specific abstraction

A second Lean package depends on Lean's metaprogramming API and, optionally, small pieces of `lean4lean`. It checks projection certificates against an elaborated target and environment snapshot. Its output is an adequacy theorem indexed by a scope and provider set.

The package should distinguish:

- `PositiveProjection`: enough information to render or fit candidates, no negative theorem;
- `NegativeAdequateProjection`: carries the one-way source-to-core theorem;
- `Refusal`: source structure could not be classified conservatively.

12.2.4 LeantVerifier: fixed-option kernel gate

A minimal process accepts a target handle and candidate source, checks the candidate with kernel checking forced on, records dependencies, and emits a signed or hashed manifest. It performs no synthesis.

12.2.5 Artifact replay

CI or an explicit `:synth -certify` mode writes a self-contained directory:

Listing 12.1: Proposed artifact bundle.

```
result/
  Candidate.lean
  Candidate.olean
  manifest.json
  core-positive.cert       # when available
  projection.cert          # for certified negative scope
  countermodel.cert        # or refutation.cert
  transcript.txt           # untrusted diagnostic trace
```

The bundle can be checked without rerunning search.

12.3 Data-flow protocol

A robust protocol is request/response oriented:

1. Leant asks the serializer for an elaborated target handle and proposed projection.
2. The projection checker returns either certified adequacy, positive-only data, or refusal.
3. Djex searches the core goal and emits candidate or negative payloads.
4. Positive core payloads are checked by `DjexCert`; rendered Lean terms are then checked by `LeantVerifier`.
5. Negative payloads are checked by `DjexCert`; only if a negative-adequate projection exists may Leant combine the theorems and display a certified scoped verdict.
6. Every displayed result includes a strength label and an artifact ID.

12.4 Trusted computing base

For a positive Lean result, the unavoidable trusted base includes the selected Lean parser/elaborator/kernel binary, imported axioms, and operating-system integrity. The synthesizer and renderer are excluded.

For a core negative result, the trusted base can be smaller:

- the Lean kernel checking `DjexCert`;
- the definitions and proofs in `DjexCert`;
- the certificate parser/checker;
- the canonical formula supplied to the checker.

For a lifted Leant negative result, add the projection checker and its environment interface. The Haskell search engine remains excluded.

12.5 Certificate identity and replay safety

Every certificate must bind to the exact problem it proves. Use cryptographic hashes or structural digests for accidental mismatch detection, but never treat a hash as proof of structural equality without comparing the corresponding data.

The manifest should bind:

- core formula and context;
- original target expression serialization;
- environment/module fingerprint;
- allowed provider set and scope flags;
- checker schema and theorem package versions;
- result artifact hashes.

A stale certificate from a previous session or environment must be rejected even when the pretty-printed goal looks identical.

12.6 Failure semantics

The system should be fail-closed for claims and fail-open for exploration:

- certificate parse/check failure: candidate may still be sent to Lean for ordinary positive verification, but no core certificate label is shown;
- verifier unavailable: show unverified engine output only in an explicitly diagnostic mode, never as a verified candidate;
- projection proof unavailable: allow positive search, disable negative certification;
- countermodel generation failure: report no certificate, not a refutation;
- replay checker disagreement: quarantine the artifact and display the disagreement prominently.

This policy preserves the interactive usefulness of Leant while making strong labels dependable.

Chapter 13

Concrete Lean specification skeleton

This chapter gives a plausible interface for the first formal package. It is intentionally small and avoids committing to every implementation detail. The listings are design-level Lean 4 sketches; they were not compiled as part of producing this report because the execution environment did not contain a Lean toolchain.

13.1 Core syntax and typing

Listing 13.1: Core definitions.

```
namespace DjexCert

abbrev Atom := Nat

inductive Formula where
  | atom : Atom -> Formula
  | bot  : Formula
  | top  : Formula
  | and  : Formula -> Formula -> Formula
  | or   : Formula -> Formula -> Formula
  | imp  : Formula -> Formula -> Formula
  deriving DecidableEq, Repr

inductive Term where
  | var    : Nat -> Term
  | lam    : Term -> Term
  | app    : Term -> Term -> Term
  | triv   : Term
  | pair   : Term -> Term -> Term
  | fst    : Term -> Term
  | snd    : Term -> Term
  | inl    : Term -> Term
  | inr    : Term -> Term
  | cases  : Term -> Term -> Term -> Term
  | abort  : Term -> Term
  deriving DecidableEq, Repr

abbrev Context := List Formula

inductive HasType : Context -> Term -> Formula -> Prop where
```

```
  | var (h : ctx[n]? = some a) : HasType ctx (.var n) a
  | lam (h : HasType (a :: ctx) body b) :
      HasType ctx (.lam body) (.imp a b)
  | app (hf : HasType ctx f (.imp a b))
        (hx : HasType ctx x a) :
      HasType ctx (.app f x) b
  | triv : HasType ctx .triv .top
  | pair (ha : HasType ctx x a) (hb : HasType ctx y b) :
      HasType ctx (.pair x y) (.and a b)
  | fst (h : HasType ctx p (.and a b)) :
      HasType ctx (.fst p) a
  | snd (h : HasType ctx p (.and a b)) :
      HasType ctx (.snd p) b
  | inl (h : HasType ctx x a) :
      HasType ctx (.inl x) (.or a b)
  | inr (h : HasType ctx y b) :
      HasType ctx (.inr y) (.or a b)
  | cases (hs : HasType ctx s (.or a b))
      (hl : HasType (a :: ctx) l c)
      (hr : HasType (b :: ctx) r c) :
      HasType ctx (.cases s l r) c
  | abort (h : HasType ctx x .bot) :
      HasType ctx (.abort x) a

end DjexCert
```

The actual implementation should add lifting and substitution lemmas early. These will be needed by both the checker proof and sequent-to-natural-deduction translation.

13.2 Annotated executable checker

The smallest checker can require annotations on eliminators and ambiguous introductions:

Listing 13.2: A deliberately simple trusted checker interface.

```
namespace DjexCert

inductive CheckedTerm where
  | var    : Nat -> CheckedTerm
  | lam    : Formula -> CheckedTerm -> CheckedTerm
  | app    : Formula -> Formula -> CheckedTerm -> CheckedTerm -> CheckedTerm
  | triv   : CheckedTerm
  | pair   : CheckedTerm -> CheckedTerm -> CheckedTerm
  | fst    : Formula -> Formula -> CheckedTerm -> CheckedTerm
  | snd    : Formula -> Formula -> CheckedTerm -> CheckedTerm
  | inl    : Formula -> CheckedTerm -> CheckedTerm
  | inr    : Formula -> CheckedTerm -> CheckedTerm
  | cases  : Formula -> Formula -> Formula ->
             CheckedTerm -> CheckedTerm -> CheckedTerm -> CheckedTerm
  | abort  : Formula -> CheckedTerm -> CheckedTerm

-- Returns an erased raw term when all annotations are consistent.
def check : Context -> Formula -> CheckedTerm -> Option Term

/-- The foundational positive-certificate theorem. -/
```

```
theorem check_sound
    (h : check ctx expected cert = some term) :
    HasType ctx term expected

end DjexCert
```

Haskell can emit the annotations. If they are wrong, checking fails. This design avoids trusting a unifier in the first trusted base. A compact unannotated certificate and a proved elaborator can be added later.

13.3 Kripke semantics

Listing 13.3: Finite Kripke countermodel interface.

```
namespace DjexCert

structure FiniteKripke (nAtoms : Nat) where
  nWorlds : Nat
  le      : Fin nWorlds -> Fin nWorlds -> Bool
  atom    : Fin nWorlds -> Fin nAtoms -> Bool
  le_refl  : forall w, le w w = true
  le_trans : forall u v w,
    le u v = true -> le v w = true -> le u w = true
  atom_mono : forall u v p,
    le u v = true -> atom u p = true -> atom v p = true

def Forces (m : FiniteKripke nAtoms)
    (w : Fin m.nWorlds) : Formula -> Prop
  | .atom p => m.atom w p = true
  | .bot    => False
  | .top    => True
  | .and a b => Forces m w a /\ Forces m w b
  | .or a b  => Forces m w a \/ Forces m w b
  | .imp a b => forall v, m.le w v = true ->
                  Forces m v a -> Forces m v b

inductive Derives : Context -> Formula -> Prop
  -- G4ip or natural-deduction/sequent rules

theorem derives_sound_kripke
    (d : Derives ctx goal) :
    forall m w,
      (forall a, a in ctx -> Forces m w a) -> Forces m w goal

structure Countermodel (ctx : Context) (goal : Formula) where
  model : FiniteKripke (atomBound ctx goal)
  root  : Fin model.nWorlds
  assumptions : forall a, a in ctx -> Forces model root a
  refutes : Not (Forces model root goal)

theorem countermodel_no_derivation
    (c : Countermodel ctx goal) :
    Not (Derives ctx goal)
```

```
end DjexCert
```

In executable certificates, the proof fields of `FiniteKripke` and `Countermodel` need not be serialized. A boolean checker validates the finite order, monotonic valuation, assumptions, and failure; a reflection theorem converts successful computation into these structures.

13.4 Reflection pattern

Listing 13.4: Certificate checking by reflection.

```
def checkCountermodel
    (ctx : Context) (goal : Formula) (raw : RawModel) : Bool :=
  checkShape raw &&
  checkPreorder raw &&
  checkMonotonicity raw &&
  ctx.all (eval raw raw.root) &&
  not (eval raw raw.root goal)

theorem checkCountermodel_sound
    (h : checkCountermodel ctx goal raw = true) :
    Not (Derives ctx goal)
```

This theorem is the trusted negative boundary. The model finder is irrelevant to its soundness.

13.5 Projection adequacy interface

A first Leant-specific formal statement may avoid full `Lean.Expr` metatheory by defining a checked structural view:

Listing 13.5: Restricted structural source view.

```
namespace LeantCert

inductive SourceView where
  | local  : FVarId -> SourceView
  | arrow  : SourceView -> SourceView -> SourceView
  | product : ProductWitness -> SourceView -> SourceView -> SourceView
  | sum     : SumWitness -> SourceView -> SourceView -> SourceView
  | top     : TopWitness -> SourceView
  | bottom  : BottomWitness -> SourceView
  | atom    : PureAtomWitness -> SourceView

structure Projection (target : Lean.Expr) where
  view : SourceView
  core : DjexCert.Formula
  matchesTarget : ViewDenotes target view
  negativeAdequate :
    forall term, RestrictedLeanTerm term target ->
      DjexCert.Derives [] core

end LeantCert
```

The first version can state `RestrictedLeanTerm` as an inductive syntax rather than all kernel expressions. Later versions can connect it to `Lean.Expr` typing and environment semantics.

13.6 Combining the theorems

The final negative theorem is tiny once its premises are available:

Listing 13.6: The lifted negative theorem.

```
theorem no_restricted_lean_term
    (p : Projection target)
    (c : DjexCert.Countermodel [] p.core) :
    Not (Exists fun term => RestrictedLeanTerm term target) := by
  intro h
  cases h with
  | intro term ht =>
    have d : DjexCert.Derives [] p.core :=
      p.negativeAdequate term ht
    exact (DjexCert.countermodel_no_derivation c) d
```

This is the central logical shape of the project. Most of the work lies in constructing the two premises honestly.

Chapter 14

Implementation roadmap

The roadmap is ordered by dependency and assurance value, not by calendar estimates.

14.1 Phase 0: freeze claim vocabulary and evidence types

Deliverables

- A short specification defining candidate, bounded miss, structural refutation, provider-relative refutation, and full-environment claim.
- Versioned result/evidence types in Haskell.
- User-facing messages updated to reflect scope.

Exit criterion

No code path can turn timeout, pruning, opaque translation, bounded instantiation, or provider discovery failure into a logically stronger result type.

14.2 Phase 1: DjexCert formula, term, and annotated checker

Deliverables

- Lean definitions for core syntax and typing.
- Canonical annotated certificate format.
- Executable checker and `check_sound` theorem.
- Port of representative Djinn regression examples.

Exit criterion

Every accepted certificate yields a theorem of declarative typing; malformed certificates are rejected by tests and mutation cases.

14.3 Phase 2: Haskell positive-certificate integration

Deliverables

- Djex emits canonical certificates for all Djinn candidates.
- A checker executable or library validates them.
- Public results distinguish internally generated from Lean-certified candidates.

Exit criterion

Disabling or corrupting the Haskell proof checker cannot cause an invalid certificate to be labeled valid by the external Lean checker.

14.4 Phase 3: finite Kripke countermodel checker

Deliverables

- Finite Kripke semantics and derivability soundness proof.
- Raw model checker and reflection theorem.
- An untrusted model generator, initially separate from LJT.
- Countermodels for representative unprovable formulas, including intuitionistically but not classically invalid cases.

Exit criterion

A Djinn negative verdict is labeled certified only when Lean accepts a countermodel or refutation certificate for the exact core sequent.

14.5 Phase 4: harden Leant's positive verifier

Deliverables

- Dedicated fixed-option verifier backend.
- Explicit forcing of kernel checking.
- Stable target/environment fingerprints.
- Axiom/dependency manifest.
- Replayable candidate module.

Exit criterion

Every displayed "kernel-verified" candidate has a replayable artifact and cannot be affected by interactive `debug.skipKernelTC` settings.

14.6 Phase 5: certified pure structural projection

Initial grammar

$$A ::= p \mid \bot \mid \top \mid A \to A \mid A \times A \mid A + A.$$

Source restrictions

- closed target modulo explicitly bound type/proposition parameters;
- no arbitrary global constants or type-class synthesis;
- recognized structural heads only;
- no recursive inductive expansion;
- no bounded rank-N hypothesis instantiation in negative mode;
- local premises mapped explicitly.

Exit criterion

Leant can combine a checked projection theorem and a checked core countermodel to report "no inhabitant in the pure structural fragment."

14.7 Phase 6: explicit provider-relative negatives

Deliverables

- Provider certificates containing exact elaborated type and core abstraction.
- Result theorem indexed by the finite provider set.
- UI and artifact manifest displaying that set.

Exit criterion

Adding or removing a provider changes the certified problem identity, and no result is described as full-environment non-inhabitance.

14.8 Phase 7: full LJT decision-procedure formalization

Deliverables

- Formal G4ip/LJT sequent calculus.
- Rule soundness, invertibility, completeness, and corrected termination proof.
- Verified decision procedure.
- Refinement relation to Haskell search states and pruning.

Exit criterion

The Lean decision procedure returns either a derivation or a proof of non-derivability for every well-formed formula, and the Haskell engine is differentially tested against it.

14.9 Phase 8: Exference partial correctness

Deliverables

- Shared type/substitution formalization.
- Contextual checker and unifier soundness.
- Constraint entailment and solver soundness.
- Scheduler termination/status theorem.

Exit criterion

Every public Exference candidate is either checked by the formal checker or by the final Lean source checker; every stopped run has an honest operational reason.

14.10 Phase 9: richer projection families

Add features only with a named theorem:

- finite non-recursive inductive schemas;
- indexed families with conservative constructor inclusion;

- restricted prenex polymorphism;
- logical-relations support for selected rank-N fragments;
- instance/provider closure for selected environments;
- strictly positive recursive families with formal fixed-point semantics.

Each extension should preserve the ability to run positive-only when negative adequacy is unavailable.

14.11 Phase 10: optional exact implementation refinement

Possible routes include:

- translate a pure Haskell subset to Lean and prove functional equivalence;
- extract the verified Lean decision procedure and compare traces;
- use property-based bisimulation between Haskell and Lean states;
- model GHC-specific operational behavior only for the small certificate producer boundary.

This phase is optional because the certificate checker already excludes the producer from the logical trusted base.

Chapter 15

Testing, CI, and adversarial validation

Formal proofs reduce the burden on tests; they do not eliminate testing. Tests remain essential for connecting Haskell producers, certificate codecs, environment fingerprints, and the compiled Lean checker.

15.1 Four complementary test layers

15.1.1 Proof-level unit examples

Inside Lean, construct small formulas, terms, and models whose outcomes are known. Examples should include:

- identity, composition, currying, product/sum associativity, and distributive plumbing;
- uninhabited atomic targets under empty context;
- $(A \to B) \to A \to B$;
- Peirce's law and excluded middle, requiring multi-world countermodels;
- formulas with top and bottom;
- malformed de Bruijn indices and branch types;
- model valuations that violate monotonicity.

The goal is to exercise reflection reductions and certificate error paths in addition to theorem statements.

15.1.2 Cross-language property tests

Generate bounded random formulas and terms in Haskell, then compare:

- existing Djinn checker acceptance;
- Lean certificate checker acceptance;
- source rendering followed by Haskell/Lean parsing;
- Djinn unbounded decision versus the verified Lean reference procedure, when available;
- countermodel evaluation in Haskell versus Lean.

Existing Djex property tests for independently checked generated proofs, identity inhabitation, budget-prefix behavior, and rendering round trips are an excellent seed corpus. They become executable refinement checks rather than the sole correctness argument.

15.1.3 Mutation testing

Every accepted artifact should be mutated systematically:

- change one atom ID;
- swap a product field or sum branch;
- increment a variable index;
- alter one constructor arity;
- delete a refutation child;
- add a cycle to a DAG;
- break Kripke monotonicity;
- change the root or provider set;
- change target or environment fingerprint;
- flip the `debug.skipKernelTC` option in the interactive session.

The checker must reject every mutation that invalidates the theorem. Mutation testing is especially good at finding fields accidentally omitted from a digest or comparison.

15.1.4 End-to-end replay tests

For each release artifact:

1. start from a clean project and pinned toolchain;
2. replay the generated `.lean` module;
3. run `leanchecker`;
4. optionally run `lean4lean`;
5. compare the axiom/dependency manifest;
6. verify all core and projection certificates without running search;
7. confirm that changing the imported environment invalidates the manifest.

15.2 Regression cases derived from historical boundaries

The repository's own audits identify high-value adversarial cases:

- a candidate is checked, then simplification changes its binding structure;
- a rendered local name captures a global constant or vice versa;
- a unifier accepts a rigid-variable escape;
- search truncation is mistaken for exhaustion;
- an approximated rank-N goal contributes negative evidence;
- an inductive family occurrence is accidentally split or merged by a display key;
- a provider-free refutation survives an unseen live provider;

- a session option disables the intended kernel gate.

Each case should have both a producer regression test and, where applicable, a certificate-level rejection test.

15.3 CI matrix

A useful CI matrix separates proof, producer, and integration concerns:

Job	Inputs	Required result
Lean formal library	Pinned Lean toolchain	All theorems compile; no sorry in trusted modules; axiom report matches allow-list
Haskell unit/property suite	Current GHC/Cabal matrix	Existing behavior and status tests pass
Certificate differential	Generated bounded corpus	Haskell and Lean checkers agree on valid/invalid certificates
Model differential	Generated formulas/models	Evaluation and monotonicity checks agree
Leant verifier hardening	Interactive option fuzzing	Kernel checking remains enabled in verifier; invalid declarations never pass
Environment scope	Projects with hidden axioms/providers	Structural negative never masquerades as full-environment negative
Replay	Generated artifact bundles	Compiler, `leanchecker`, and optional independent checker accept the exact artifacts
Compatibility	Previous schema versions	Supported versions replay; unsupported versions fail explicitly
Adversarial parser	Fuzzed certificate bytes	Deterministic rejection without resource blowup or partial acceptance

15.4 Performance tests without weakening proofs

Certificate checking should have explicit resource bounds, but a bound failure is an operational error rather than a logical result. Track:

- certificate size versus formula size;
- checker time and peak memory;
- DAG sharing ratio;
- Kripke world count;
- parser depth and integer limits;
- cost of environment/projection validation;
- official and independent replay time.

If a certificate exceeds the checker budget, Leant should retain the ordinary bounded-search message. It must not trust the producer's claim merely because verification is expensive.

Chapter 16

Risks and mitigations

Risk	Failure mode	Mitigation
Scope inflation	A theorem about a restricted calculus is described as a theorem about all Lean or Haskell terms	Index evidence by explicit language/environment scope; use precise UI labels; include provider manifest
Common-mode checking	Official Lean and a derived checker share the same conceptual omission	Add a small domain-specific checker; use structurally diverse replay; pin revisions; retain source artifacts
Mutable verifier options	Interactive `:set debug.skipKernelTC true` weakens positive checking	Dedicated verifier process; force option false locally; regression test
Pretty-print identity	Alpha-equivalent or definitionally equal atoms split, or unrelated nodes collide unexpectedly	Canonical structural IDs; checked quotient maps; compare expressions, not only hashes or strings
Opaque structure	A global proposition/type is collapsed to an atom, removing valid constructions	Positive-only fallback; head-specific certificates; conservative over-approximation
Provider incompleteness	A live constant inhabits the target but was outside the bounded inventory	Never claim full environment; certify explicit provider set or environment closure
Rank-N frontier incompleteness	Bounded instantiation misses a valid polymorphic use	Disable negative evidence unless frontier completeness or parametricity is proved
Recursive-family truncation	One-layer view omits recursive eliminations or induction	Positive-only initially; later fixed-point semantics or proved normal-form restriction
Haskell bottom	"Uninhabited Haskell type" is false under ordinary non-strict semantics	State total core-term claim; do not quantify over `undefined`, exceptions, or unsafe features

Risk	Failure mode	Mitigation
Certificate blowup	Complete refutation trees become too large	DAG sharing; prefer Kripke models; compress untrusted envelope but check canonical decompression
Parser/codec bug	Haskell and Lean interpret the same bytes differently	Canonical grammar; versioning; round-trip properties; length-delimited fields; independent parser fuzzing
Theorem/package drift	Lean upgrades change expression or environment behavior	Pin toolchains; explicit compatibility layer; artifact manifests; upgrade proofs before strengthening labels
Axiom ambiguity	Kernel-checked candidate depends on user axioms or sorry-bearing imports	Collect dependencies; display/allow-list axioms; distinguish type correctness from axiom policy
Over-verifying Haskell	Project stalls on GHC semantics before any user-visible assurance ships	Maintain certificate-first roadmap; exact refinement is optional final layer

16.1 A note on proof maintenance

The formal library should keep its logical core independent of Lean metaprogramming internals. Formula, term, typing, Kripke semantics, and certificate checkers are stable mathematics. Only the Leant projection adapter should track changes to `Lean.Expr`, inductive metadata, environment APIs, and module formats.

This separation is the main maintenance defense against Lean's rapid development.

Chapter 17

Recommended user-facing semantics

Correct labels are part of correctness. The UI should expose evidence strength rather than compress it into success/failure.

17.1 Positive labels

Lean-kernel verified candidate
The fixed-option verifier accepted a declaration at the exact target in the recorded environment.

Lean-kernel verified; dependencies listed
As above, with an explicit axiom/declaration dependency manifest.

Core-certificate checked
The corresponding Djinn/Exference core certificate passed the formally proved checker. This is supplementary when a Lean term is also checked.

Engine candidate, unverified
Diagnostic mode only; never the default display category.

17.2 Negative labels

Certified structurally uninhabited
No term exists in the pure structural calculus represented by the checked projection.

Certified uninhabited relative to providers P
No permitted term exists using only the explicitly listed provider set.

Core formula refuted
A checked countermodel/refutation establishes non-derivability of the abstract sequent, but no source adequacy theorem was available.

No candidate found within bounds
Search stopped without a logical verdict.

Positive-only projection
The goal was approximated in a way suitable for candidate generation but unsuitable for negative reasoning.

Out of certified fragment
The translator refused to make a claim because the source structure or environment could not be abstracted conservatively.

17.3 Suggested replacement for the current strongest message

Instead of

> "provably uninhabited – no closed term of this polymorphic type exists,"

use one of:

> "certified: no inhabitant exists in the pure structural term language"

or

> "certified relative to 12 listed providers: no permitted inhabitant exists"

A secondary note can explain that the result is constructive/meta-theoretic and does not prove the object's negation. If a full environment theorem or object-level refutation is actually available, it may receive a stronger label.

17.4 Machine-readable strength

The REPL output is not enough. JSON/transcript output should include:

Listing 17.1: Illustrative result strength metadata.

```
{
  "kind": "negative",
  "strength": "structural-calculus",
  "coreCertificate": "kripke-v1",
  "projectionCertificate": "lean-structural-v1",
  "scope": {
    "globalConstants": "forbidden-except-structural-heads",
    "providers": [],
    "rankN": "positive-only",
    "recursiveInductives": "positive-only"
  },
  "environmentFingerprint": "...",
  "leanRevision": "1d2dde6d..."
}
```

This makes downstream automation robust against wording changes.

Chapter 18

Prioritized theorem inventory

ID	Theorem	Statement sketch	Dependency
D1	Core weakening	Typing is preserved when the context is extended with correctly shifted variables	Syntax/typing
D2	Core substitution	Substituting a well-typed term for the newest variable preserves typing	D1
D3	Annotated checker soundness	`check ctx A c = some t` implies `HasType ctx t A`	D1–D2
D4	Haskell certificate normalization	A well-formed Haskell formula/term certificate decodes to the same normalized core object	Codec, metadata
D5	Unification soundness	Successful proof-type unification satisfies all generated equations	Substitution/occurs check
D6	Compact checker soundness	Current-style unannotated proof checking implies declarative typing	D3, D5
L1	G4ip rule soundness	Derivations of premises yield a derivation of the conclusion	Sequent calculus
L2	Rule invertibility/complete branching	A derivation of a conclusion yields derivability of required child branch(es)	L1, structural rules
L3	Termination measure	Every recursive decision call strictly decreases a well-founded measure	Corrected G4ip measure
L4	Decision soundness	A returned proof is a derivation	L1
L5	Decision completeness	A negative decision implies no derivation	L2–L3
L6	Haskell search refinement	Haskell unbounded search result corresponds to verified decision/certificate	State invariants
K1	Kripke monotonicity	Forcing is monotone along accessibility	Model definition
K2	Derivability semantic soundness	Derivable sequents hold in every Kripke model	K1, typing/rules

ID	Theorem	Statement sketch	Dependency
K3	Countermodel checker soundness	Accepted raw model implies no derivation	K2, reflection
E1	Shared type substitution	Capture-avoiding substitution preserves scope/kinding	Type syntax
E2	Exference unifier soundness	Solved equations hold and rigid variables remain rigid	E1
E3	Constraint solver soundness	Every discharged class constraint is derivable from declared instances/premises	Instance relation
E4	Expression checker soundness	Accepted expression has the claimed type and no escaped rigid obligations	E1–E3
E5	Scheduler status honesty	Each completion status has its documented operational witness	Finite transition model
P1	Structural-head recognition	Checked Lean heads implement the corresponding core connective abstraction	Lean environment adapter
P2	Atom quotient safety	Every source atom occurrence maps consistently; deliberate merges preserve derivability	Canonical IDs, substitution
P3	Pure projection adequacy	Every restricted source inhabitant maps to a core derivation	P1–P2
P4	Finite-inductive adequacy	Every value of a checked non-recursive family maps to a sum-of-products derivation	Constructor schema
P5	Provider adequacy	Every use of a listed provider maps to the corresponding core premise	Elaborated provider type
V1	Fixed verifier soundness	Accepted artifact was kernel checked with required options and exact target/environment	Verifier protocol
V2	Dependency manifest soundness	Listed axioms/declarations cover the accepted candidate's dependencies	Environment traversal
C1	Structural negative lift	Core refutation plus P3 implies no restricted Lean inhabitant	K3 or L5, P3
C2	Provider-relative negative lift	Core refutation plus P3/P5 implies no inhabitant using provider set P	K3 or L5, P3/P5

Chapter 19

Source-to-obligation map

19.1 Djex modules

Source module	Formalization relevance
`synthesis/.../Type.hs`	Shared type grammar, substitution, validation, rigid/flexible distinction, forall and constraint syntax
`Djinn/Internal/LJTFormula.hs`	Formula and proof-term grammar, metadata validation, source of canonical certificate mapping
`Djinn/Internal/ProofCheck.hs`	Independent checker, proof-type metavariables, unification, occurs check; highest-priority proof target
`Djinn/Internal/LJT.hs`	Actual search rules, state organization, pruning, scheduling, and budget semantics
`Djinn/Core.hs`	Translation plans, negative-evidence flags, search completion, internal checking, and public evidence classification
`Exference/Core/ExpressionCheck.hs`	Contextual candidate checker, deconstructor typing, rigid provenance, residual constraints
`Exference/Core/Internal/SearchControl.hs`	Branch scheduling, truncation, and identifier allocation
`Exference/Core/Internal/Exference.hs`	Public validated-candidate boundary and rich completion status
`djinn/test-property/PropertySpec.hs`	Existing randomized regression corpus to reuse for checker/refinement testing

19.2 Leant modules

Source module	Formalization relevance
`Leant/Synth/Fragment.hs`	Lean meta serializer, structural grammar, atom safety, fuel/depth, inductive schemas, current projection completeness predicate
`Leant/Synth/Engine.hs`	Engine routing, refutation strength, projection/family completeness gating, Exference non-refutation policy

Source module	Formalization relevance
`Leant/Synth/Render.hs`	Positive-only renderer variants and the boundary at which textual ambiguity is delegated to Lean
`src/Main.hs`	Goal translation, provider lanes, timeout semantics, classical fallback, candidate verification, display, and current strongest negative wording

19.3 Lean modules and adjacent projects

Source	Formalization relevance
`Lean/AddDecl.lean`	Exact kernel-check option boundary and sorry warning behavior
`LeanChecker.lean`	Official declaration replay; useful second path but not an external verifier
`lean4lean`	Reified Lean expressions/environments/type checking and verification infrastructure; optional independent replay and reusable metatheory
LeanSAT / `Std.Tactic.BVDecide`	Established pattern of untrusted high-performance producer plus verified certificate checker
Lean issue #14576 and PR #14577	Recent example motivating revision pinning, fixed verifier options, and structurally diverse checking

Chapter 20

Conclusions

The possibility investigated in this report is real and technically promising. A formalization does not need to wait for a complete semantics of GHC Haskell or a full normalization theorem for arbitrary Lean environments. The repositories already contain the right seams for a certificate-oriented design.

The most important conclusions are:

1. **Positive correctness can be made very strong immediately.** Harden Leant's existing kernel gate, force kernel checking on, persist declarations, and report dependencies. Search and rendering remain untrusted.
2. **Djinn's independent proof checker is the best first formal theorem.** Its soundness is small, useful, and directly connected to current implementation behavior.
3. **Negative IPC results should be certificate based.** A finite Kripke countermodel checked in Lean can establish non-derivability without proving the exact Haskell search complete. A complete LJT refutation DAG is a complementary format.
4. **The hard part of Leant negatives is the source abstraction theorem.** One-way over-approximation is sufficient; full equivalence is not. This makes a pure structural fragment attainable.
5. **Environment scope must be explicit.** Provider-free or bounded-provider search cannot justify an unrestricted claim about every term in a live Lean environment. Formal result types and UI wording should expose the structural or provider-relative scope.
6. **Rank-N, recursive-inductive, and type-class features should stay positive-only until their adequacy theorems exist.** The current design already tends in this direction; proof-carrying projection evidence will make the boundary exact.
7. **Verifying the exact Haskell implementation is optional, not foundational.** Once certificates are checked, producer bugs cause rejected or missing results rather than false theorems.

The proposed architecture therefore supports a sequence of genuine formal wins: first checked core witnesses, then checked countermodels, then a certified Lean structural projection, and finally richer fragments. Each stage strengthens the product without making later research a prerequisite for present trustworthiness.

Appendix A

Minimal first pull-request specification

A narrowly scoped first formalization contribution could contain the following.

A.1 Lean files

- `DjexCert/Formula.lean`: binary formula grammar and decidable equality.
- `DjexCert/Term.lean`: raw and annotated de Bruijn terms.
- `DjexCert/Typing.lean`: declarative typing plus weakening/substitution.
- `DjexCert/Check.lean`: executable annotated checker.
- `DjexCert/CheckSound.lean`: proof of checker soundness.
- `DjexCert/Codec.lean`: simple S-expression or JSON-subset parser for certificates.
- `DjexCert/Examples.lean`: hand-written examples and malformed cases.

A.2 Haskell files

- a canonical formula normalization module;
- a de Bruijn conversion module for checked Djinn terms;
- a certificate encoder with explicit schema version;
- an integration test invoking the Lean checker on generated examples.

A.3 Non-goals

The first contribution does not need:

- LJT completeness or termination;
- Kripke semantics;
- Leant projection formalization;
- rank-N types or constraints;
- exact source rendering equivalence;
- GHC refinement.

Its single theorem is already valuable: accepted Djinn certificates denote well-typed core proof terms.

Appendix B

Countermodel certificate example

Consider excluded middle over one atom, $p \vee (p \to \bot)$. A one-world classical valuation cannot refute it. A two-world Kripke model can:

- worlds $w_0 \leq w_1$;
- p is false at w_0 and true at w_1;
- at w_0, p is not forced;
- at w_0, $p \to \bot$ is not forced because the extension w_1 forces p but not $\bot$;
- therefore the disjunction is not forced at w_0.

A raw certificate could be:

Listing B.1: Illustrative countermodel payload.

```
(model
  (worlds 2)
  (order (0 0) (0 1) (1 1))
  (atoms
    (p 0 false)
    (p 1 true))
  (root 0))
```

The checker validates reflexivity/transitivity, monotonicity of p, and recursive forcing. The certificate then proves that the formula has no derivation in IPC, while correctly avoiding the stronger and generally unavailable object-level claim $\neg(p \vee \neg p)$.

Appendix C

Result-type sketch for Haskell

A Haskell API can mirror proof strength without pretending that Haskell values themselves are proofs:

Listing C.1: Illustrative evidence-aware result type.

```
data PositiveEvidence
  = EngineChecked CoreCertificate
  | LeanKernelChecked LeanArtifact

data NegativeScope
  = PureStructural
  | ExplicitProviders [ProviderId]
  | ClosedEnvironment EnvironmentFingerprint

data NegativeEvidence
  = CoreCountermodel CoreFormula CountermodelCertificate
  | CoreRefutation   CoreFormula RefutationCertificate
  | LiftedNegative  NegativeScope ProjectionCertificate
                    CoreNegativeCertificate

data SynthesisResult
  = Candidates [Candidate] [PositiveEvidence]
  | CertifiedNegative NegativeEvidence
  | NoTermFound [SearchLimit] [ApproximationReason]
  | Refused RefusalReason
  | EngineFailure Text
```

Constructors carrying certified evidence should be private to modules that have invoked the external checker and validated artifact identity.

Appendix D

Bibliographic and repository references

Bibliography

[1] Vladimir Reshetnikov. *Djex repository*, revision `f50b5eac86726c7530d95e48c27101b7c0fa25c9`, 10 August 2026. https://github.com/VladimirReshetnikov/Djex/tree/f50b5eac86726c7530d95e48c27101b7c0fa25c9

[2] Vladimir Reshetnikov. *Leant repository*, revision `0160237e4bdf6e342a55f5a86ce5d5292728024f`, 10 August 2026. https://github.com/VladimirReshetnikov/Leant/tree/0160237e4bdf6e342a55f5a86ce5d5292728024f

[3] Lean developers. *Lean 4 repository*, revision `1d2dde6d64cb3da989bab118f4d386eb55237fe5`, 10 August 2026. https://github.com/leanprover/lean4/tree/1d2dde6d64cb3da989bab118f4d386eb55237fe5

[4] Lean developers. *Lean/AddDecl.lean*, checked declaration and `debug.skipKernelTC` boundary, revision `1d2dde6d64cb3da989bab118f4d386eb55237fe5`. https://github.com/leanprover/lean4/blob/1d2dde6d64cb3da989bab118f4d386eb55237fe5/src/Lean/AddDecl.lean

[5] Lean developers. *LeanChecker.lean*, declaration replay utility, revision `1d2dde6d64cb3da989bab118f4d386eb55237fe5`. https://github.com/leanprover/lean4/blob/1d2dde6d64cb3da989bab118f4d386eb55237fe5/src/LeanChecker.lean

[6] Mario Carneiro and contributors. *lean4lean*, revision `1a16b72d2e35932a82aa501beb29ef2c3d072580`, 5 August 2026. https://github.com/digama0/lean4lean/tree/1a16b72d2e35932a82aa501beb29ef2c3d072580

[7] Lean developers. *LeanSAT: verified SAT reasoning and LRAT certificate checking*. The project is archived after integration into Lean core as `Std.Tactic.BVDecide`. https://github.com/leanprover/leansat

[8] Roy Dyckhoff. "Contraction-Free Sequent Calculi for Intuitionistic Logic." *Journal of Symbolic Logic*, 57(3):795–807, 1992. DOI: 10.2307/2275431.

[9] Roy Dyckhoff. "Contraction-Free Sequent Calculi for Intuitionistic Logic: A Correction." *Journal of Symbolic Logic*, 83(4):1680–1682, 2018. DOI: 10.1017/jsl.2018.38.

[10] Roy Dyckhoff and Sara Negri. "Admissibility of Structural Rules for Contraction-Free Systems of Intuitionistic Logic." *Journal of Symbolic Logic*, 65(4):1499–1518, 2000. DOI: 10.2307/2695061.

[11] A. S. Troelstra and D. van Dalen. *Constructivism in Mathematics: An Introduction*, Volume I. North-Holland, 1988.

[12] Morten Heine Sørensen and Paweł Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Elsevier, 2006.

[13] John C. Reynolds. “Types, Abstraction and Parametric Polymorphism.” In *Information Processing 83*, pages 513–523, 1983.

[14] Lean issue #14576. “Kernel accepts wrong-structure projections, allowing an axiom-free proof of False,” opened and closed 28 July 2026. https://github.com/leanprover/lean4/issues/14576

[15] Lean pull request #14577. “fix: missing check at kernel inductive declaration,” merged 28 July 2026, merge commit `a39eab69e1eee9ad38f4efe507907b1026a77808`. https://github.com/leanprover/lean4/pull/14577